ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Modernizing C++ Legacy Systems with Rust: Strategies for Safe Interoperability and Performance Evaluation in Rust-Based Boost Components

Filipe Daniel Ferreira

Dissertation submitted in partial fulfilment of the requirements for the Master's Degree in Critical Computing Systems Engineering

Supervisor: Luis Miguel Pinho

President:

Ricardo Severino, Instituto Superior de Engenharia do Porto

Members:

Tiago Carvalho, Faculdade de Engenharia da Universidade do Porto

Luis Miguel Pinho, Instituto Superior de Engenharia do Porto

Porto, October 12, 2026

Statement of Integrity

I hereby declare having conducted this academic work with integrity. I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

I declare that the work presented in this document is original and my own, and has not previously been used for any other purpose.

I further declare that I have fully followed the Code of Good Practices and Conduct of the Polytechnic Institute of Porto.

ISEP, Porto, October 12, 2026

Abstract

The growing complexity of safety-critical software systems, particularly in the automotive domain, has intensified concerns regarding memory safety, concurrency correctness, and long-term maintainability of large C++ codebases. Although C++ remains the dominant language in embedded and real-time systems due to its performance and mature ecosystem, its reliance on manual memory management continues to be a major source of vulnerabilities. Rust has emerged as a promising alternative, offering strong compile-time guarantees for memory safety and data-race freedom without introducing garbage collection. However, the adoption of Rust in industrial environments is hindered by the scale of existing C++ systems and their deep dependence on template-heavy frameworks such as the Boost C++ Libraries.

This dissertation investigates strategies for incrementally modernizing legacy C++ systems by integrating Rust in a safe and performant manner. Focusing on Boost-based components commonly used in automotive software architectures, it proposes a systematic methodology for selecting migration candidates, designing bidirectional interoperability interfaces, and re-implementing critical functionality in Rust. The work explores Foreign Function Interface (FFI) patterns, evaluates existing interoperability tools, and addresses the architectural challenges arising from the mismatch between C++ templates and Rust's ownership model.

A Rust-based reimplementation of selected Boost components is developed and integrated into a hybrid C++/Rust system. Performance, memory behaviour, and interoperability overhead are experimentally evaluated for the selected workloads. The results provide evidence that the migrated functionality can preserve the tested observable behaviour while remaining interoperable with existing C++ code. The experiments also expose workload-specific trade-offs in execution time and memory behaviour, providing practical guidance for incremental Rust adoption in high-assurance software systems.

Keywords: Boost, C++, Rust, interoperability, migration, memory safety

Resumo

A crescente complexidade dos sistemas de software crítico e seguro, particularmente no domínio automóvel, intensificou as preocupações relativas à segurança da memória, à correção da concorrência e à manutenção a longo prazo de grandes bases de código C++. Embora o C++ continue a ser a linguagem dominante em sistemas embebidos e de tempo real devido ao seu desempenho e ecossistema maduro, a sua dependência da gestão manual da memória continua a ser uma importante fonte de vulnerabilidades. O Rust surgiu como uma alternativa promissora, oferecendo fortes garantias em tempo de compilação para a segurança da memória e a ausência de conflitos de dados, sem introduzir a recolha de lixo. No entanto, a adoção do Rust em ambientes industriais é dificultada pela escala dos sistemas C++ existentes e sua profunda dependência de frameworks fortemente baseadas em modelos, como as bibliotecas Boost C++.

Esta dissertação investiga estratégias para modernizar incrementalmente sistemas C++ antigos, integrando Rust de maneira segura e com bom desempenho. Com foco em componentes baseados em Boost comumente usados em arquiteturas de software automotivo, ela propõe uma metodologia sistemática para selecionar candidatos à migração, projetar interfaces de interoperabilidade bidirecionais e reimplementar funcionalidades críticas em Rust. O trabalho explora padrões de FFI, avalia ferramentas de interoperabilidade existentes e aborda os desafios arquitetônicos decorrentes da incompatibilidade entre modelos C++ e o modelo de propriedade do Rust.

Uma reimplementação baseada em Rust de componentes Boost selecionados é desenvolvida e integrada a um sistema híbrido C++/Rust. O desempenho, o comportamento da memória e a sobrecarga de interoperabilidade são avaliados experimentalmente para as cargas de trabalho selecionadas. Os resultados fornecem evidência de que as funcionalidades migradas podem preservar o comportamento observável testado, mantendo a interoperabilidade com código C++ existente. As experiências também evidenciam compromissos específicos das cargas de trabalho relativamente ao tempo de execução e ao comportamento da memória, fornecendo orientações práticas para a adoção incremental de Rust em sistemas de software de alta garantia.

Contents

1 Introduction **1**
1.1 High-Level Overview and Motivation . . . 1
1.2 Problem Statement . . . 2
1.3 Research Questions . . . 3
1.4 Research Objectives . . . 3
1.5 Research Contributions . . . 4
1.6 Dissertation Structure . . . 4
1.7 Privacy and ethics . . . 5
1.8 Use of AI Tools . . . 5

2 State of the art **7**
2.1 Rust . . . 7
2.1.1 Overview . . . 7
Ownership and Memory Safety . . . 8
Tooling and ecosystem . . . 9
Adoption and Industrial Applications . . . 9
2.2 C++ and the Boost Library . . . 10
2.2.1 C++ . . . 10
2.2.2 Boost C++ Libraries . . . 12
2.2.3 The Boost Paradox: Barrier and Bridge . . . 12
2.3 Interoperability between Rust and C++ . . . 16
2.3.1 Undefined Behavior and Vulnerability Mitigation . . . 16
2.3.2 The Safety Air Gap in Hybrid Architectures . . . 16
2.3.3 Systematic Porting vs. the Rewrite Trap . . . 17
2.3.4 Foreign Function Interface (FFI) . . . 18
2.3.5 Interoperability tools . . . 18
2.3.6 Evaluation Criteria for Interoperability Mechanisms . . . 18
2.3.7 Boost migration for Rust . . . 19
2.4 Case studies . . . 21
2.4.1 Concurrency Safety: Migrating Locking primitives . . . 21
2.4.2 “Just Use Rust”: A Best-Case Historical Study of Open Source Vulnerabilities in C . . . 22

3 Methodology and Solution Proposal **25**
3.1 Component Selection Criteria . . . 25
3.1.1 Automotive-Oriented Selection Rationale . . . 25
3.1.2 Operational Selection Criteria and Assessment . . . 26
3.1.3 Boost.Align and Boost.Endian as Migration Enablers . . . 28
3.1.4 Deprioritized Components . . . 29
3.1.5 Safety-Critical Constraints and ISO 26262 Alignment . . . 29

3.1.6 Boost Libraries Relevant to Eclipse Safety-Critical Open Research Environment (SCORE) . . . 29
3.1.7 Comparative Analysis as a Migration Enabler . . . 30
3.1.8 From Library Gaps to New Rust Automotive Crates . . . 31
3.1.9 Automotive Safety Integrity Level (ASIL)-Oriented Migration Strategy . . . 31
3.1.10 Migration Workflow Overview . . . 32
3.2 Research Design and Evaluation Plan . . . 33
3.2.1 Research Design . . . 33
3.2.2 Development Workflow . . . 34
3.2.3 Validation and Evaluation Strategy . . . 34
3.2.4 Experimental Configuration . . . 34
3.2.5 Evaluation Scope and Limitations . . . 35

4 Migration Strategy **37**

5 Selected Libraries **43**
5.1 Boost.Align . . . 43
5.1.1 Overview . . . 43
5.1.2 Functionality . . . 44
5.1.3 Relevance to automotive software . . . 45
5.1.4 Challenges of reimplementation in Rust . . . 46
5.1.5 Rationale for selection . . . 47
5.2 Selection Summary and Migration-Enabling Patterns . . . 47
5.3 Boost.Endian . . . 48
5.3.1 Overview . . . 48
5.3.2 Functionality . . . 49
Endian conversion functions . . . 49
Endian buffer types . . . 50
Endian arithmetic types . . . 50
5.3.3 Binary data portability . . . 50
5.3.4 Relevance to automotive software . . . 51
5.3.5 Memory representation and portability . . . 52
5.3.6 Performance considerations . . . 52
5.3.7 Challenges of reimplementation in Rust . . . 53
5.3.8 Rationale for selection . . . 53

6 Implementation, Evaluation and Critical Analysis **55**
6.1 Implementation Overview . . . 55
6.2 Implementation Environment . . . 55
6.3 Boost.Align Reimplementation . . . 57
6.3.1 Scope and API mapping . . . 57
6.3.2 FFI Design and C ABI Selection . . . 58
6.3.3 Alignment representation . . . 60
6.3.4 Aligned Memory Allocation . . . 62
6.3.5 Pointer Alignment . . . 63
6.3.6 Allocator Support . . . 64
6.3.7 Unsafe Code Isolation . . . 66
6.3.8 Boost.Align Test Suite . . . 68
6.4 Boost.Endian Reimplementation . . . 70
6.4.1 Scope and API Mapping . . . 71

6.4.2 Endian Representation . . . 73
6.4.3 Endian Conversion Functions . . . 74
Load and Store Operations . . . 76
24-bit Values . . . 77
6.4.4 Endian Buffer Types . . . 78
Equality and Ordering . . . 80
6.4.5 Endian Arithmetic Types . . . 80
Arithmetic Operators . . . 81
Comparison and Ordering . . . 83
Conversion Hoisting . . . 83
6.4.6 C++ Interoperability . . . 84
Conversion Interface . . . 85
Buffer Interface . . . 86
Arithmetic Interface . . . 86
6.4.7 Unsafe Code isolation . . . 87
6.4.8 Boost.Endian Test Suite . . . 88
Conversion Tests . . . 88
Buffer Tests . . . 90
Arithmetic Tests . . . 90
C++ Integration Tests . . . 91
FFI Overhead Benchmark . . . 92
6.5 Evaluation and Critical Analysis . . . 94
6.5.1 Evaluation Methodology . . . 94
6.5.2 Boost.Align Performance Evaluation . . . 94
6.5.3 Boost.Align FFI Overhead . . . 95
6.5.4 Boost.Endian Performance Evaluation . . . 96
6.5.5 Unsafe Code Inventory . . . 97
6.5.6 Safety Guarantees Across the FFI Boundary . . . 99
6.5.7 Generalizable and Case-Specific Migration Decisions . . . 100
6.5.8 Critical Analysis of the Contribution . . . 101
6.5.9 Limitations of the Evaluation . . . 102
6.5.10 Implementation Summary . . . 102
Important Scope Note . . . 104

7 Conclusions and Future Work 105
7.1 Conclusions . . . 105
7.2 Answers to the Research Questions . . . 106
7.2.1 RQ1 – Architectural and interoperability patterns . . . 106
7.2.2 RQ2 – Component-selection criteria . . . 106
7.2.3 RQ3 – Rust reimplementation while preserving behaviour . . . 106
7.2.4 RQ4 – FFI boundary design . . . 106
7.2.5 RQ5 – Observed performance characteristics . . . 106
7.2.6 RQ6 – Safety guarantees and FFI contracts . . . 107
7.3 Contributions . . . 107
7.4 Limitations . . . 107
7.5 Future Work . . . 108
7.5.1 Implementation Completion . . . 108
7.5.2 Stronger Validation . . . 108
7.5.3 FFI and Integration Refinement . . . 109
7.5.4 Automotive-Oriented Validation . . . 109

7.5.5 Extension to Additional Boost Components 109
7.5.6 Summary 110

Bibliography **111**

List of Figures

2.1 Concrat 22
2.2 rustpreventions 23

3.1 Boost-to-Rust Migration Workflow 33

4.1 Strategy 39
4.2 Migration phases 40

6.1 Measured Boost.Align end-to-end execution time 95
6.2 Measured Boost.Align throughput 96
6.3 Selected Boost.Align FFI-overhead measurements; logarithmic vertical scale . 97
6.4 Boost.Endian measured execution time across input sizes 98

List of Tables

2.1 Qualitative comparison of Rust/C++ interoperability mechanisms for the migration scenario . . . 19
2.2 Rust ecosystem crates that replace common Boost C++ libraries. . . . 20
2.3 Top 20 Most Common Vulnerabilities and Rust Prevention Status (Meneely et al. 2025) . . . 24

3.1 Operational criteria used for Boost component selection . . . 27
3.2 Qualitative assessment of candidate Boost libraries . . . 27
3.3 Boost libraries relevant to Eclipse SCORE–style automotive systems and Rust equivalents . . . 30

5.1 Migration-enabling patterns exposed by the selected libraries . . . 48

6.1 Implementation environment . . . 57
6.2 Mapping of Boost.Align concepts to the Rust implementation . . . 58
6.3 Mapping between Boost.Endian concepts and their Rust implementation. . . . 72
6.4 Boost.Align end-to-end benchmark results . . . 94
6.5 Observed memory behaviour in the Boost.Align benchmark . . . 95
6.6 Selected Boost.Align FFI-overhead benchmark results . . . 96
6.7 Boost.Endian FFI benchmark results . . . 97
6.8 Measured unsafe-code inventory in the Rust implementation and tests . . . 99
6.9 Safety properties inside Rust and at the FFI boundary . . . 100
6.10 Generalizable and case-specific migration decisions . . . 101

List of Acronyms

ASIL	Automotive Safety Integrity Level.
CWE	Common Weakness Enumeration.
ECU	Electronic Control Unit.
FFI	Foreign Function Interface.
QM	Quality Management.
RAII	Resource Acquisition Is Initialization.
SCORE	Safety-Critical Open Research Environment.
SDV	Software-Defined Vehicle.
STD	Standard C++ Library.
TMP	Template Metaprogramming.
WCET	Worst-Case Execution Time.

Chapter 1

Introduction

1.1 High-Level Overview and Motivation

The automotive industry is performing a profound transformation driven by the rise of the Software-Defined Vehicle (SDV), autonomous driving technologies and increasingly interconnected Electronic Control Unit (ECU) (S&P Global Mobility 2025). With the growth of these systems in complexity and responsibility, the reliability of high-assurance software becomes critical. The systems must operate under strict real-time, reliability and safety requirements regulated by standards as ISO 26262 (Deloitte 2024). In this environment, memory safety has a special importance, as software faults can directly translate into physical hazards (Bristol 2025).

Historically, C++ has been the dominant language (paired with C) in embedded and automotive development due to its performance, predictable resource use and mature ecosystem (*ISO 26262-6:2018 Road vehicles — Functional safety — Part 6: Product development at the software level* 2018). However, its dependence on manual memory management continues to be a major source of vulnerabilities such as: buffer overflows, dangling pointers, race conditions and undefined behaviors (The MISRA Consortium 2023). Such issues are incompatible with the needs of safety critical cyber-physical systems, where a minor software defect can lead to serious consequences (AUTOSAR Partnership 2023).

Rust has rapidly emerged as a plausible successor for safety-critical software. Enforcing memory safety, ownership and data race free concurrency at compile time (*Rust Documentation* 2025). Rust promises to drastically reduce classes of defects endemic to C/C++ systems (Blandy, Orendorff, and Tindall 2021). Rust achieves this without introducing garbage collection, which can support predictable resource management in systems with real-time requirements (*Rust Documentation* 2025).

Despite the advantages Rust presents, the transition from C++ to Rust in large industrial ecosystems remains complex. Automotive software contains millions of lines of validated legacy C++ code. This code is often built at top of complex frameworks and libraries as Boost C++ libraries (Tolnay 2025). Rewriting those systems from scratch is infeasible. To avoid

it, the companies usually pursue incremental and hybrid migration. This migration requires robust interoperability between Rust and C++ (Google Android Team 2024).

This dissertation is motivated byt the need to address that gap: enabling safe, practical and measurable integration of Rust into deeply entrenched C++ based automotive architectures, specially real-time related architectures.

Rewriting millions of lines of validated, legacy C++ code is operationally infeasible, necessitating a hybrid integration strategy. This transition is particularly problematic for automotive subsystems (such as sensor fusion, V2X communication and path planning) that are deeply coupled with the C++ ecosystem and reliant on complex, template-heavy frameworks like the Boost C++ Libraries.

1.2 Problem Statement

The central results from the tension between two different realities:

1. **C++ is deeply entrenched in automotive systems**, specially through heavily templated, performance-critical frameworks like Boost.
2. **Rust provides compile-time ownership and borrowing checks**, but its adoption is hindered by the difficulty of interoperating with complex C++ abstractions creating a challenge for a transition between C++ and Rust.

Currently, the interoperability approaches rely mainly on C-style Foreign Function Interface (FFI) . However, it is not possible to natively represent C++ templates, ownership semantics, RAII lifecycles or advanced Boost abstractions with C FFI (Google 2021).

As result of this limitation:

- The integration often requires unsafe code, reducing Rust's safety benefits.
- Interfacing with template-based heavily libraries, as Boost.Geometry and Boost.Asio, introduces high engineering overhead.
- Crossing the Rust and C++ boundary imposes a performance cost that is poorly characterized for high-assurance systems.
- The reviewed literature does not provide a sufficiently detailed methodology for the systematic and incremental migration of C++ Boost components to Rust while maintaining compatibility with automotive subsystems.

The problem is not simply porting code from C++ to Rust, but the absence of validated techniques, architectural patterns, and tooling to bridge the impedance mismatch between C++ templates and Rust's ownership model, without changing the required performance and safety properties.

1.3 Research Questions

To address the research gap, this dissertation is guided by the following research questions:

1. **RQ1:** What architectural and interoperability patterns can support the incremental migration of selected Boost components from C++ to Rust?
2. **RQ2:** What criteria can be used to identify Boost components that are suitable candidates for incremental migration in automotive-oriented software?
3. **RQ3:** How can selected Boost components be reimplemented using Rust abstractions while preserving their relevant observable behaviour?
4. **RQ4:** How can Rust/C++ FFI boundaries be designed to explicitly manage ownership, representation, lifetime, and error-handling constraints?
5. **RQ5:** What performance characteristics are observed when the selected Rust implementations are compared with their C++ counterparts under the evaluated workloads?
6. **RQ6:** Which safety guarantees are provided by Rust within the migrated components, and which guarantees must remain explicit contracts at the FFI boundary?

The questions are intentionally scoped to the selected case studies and evaluated workloads. In particular, the performance experiments measure observed execution time and throughput rather than worst-case execution time, and therefore do not establish hard real-time guarantees.

1.4 Research Objectives

The primary objective of this dissertation is to design, implement and evaluate a methodology for the incremental migration of selected Boost C++ components into Rust, while maintaining controlled interoperability with existing C++ systems. The work focuses on identifying repeatable migration decisions, applying them to two representative low-level libraries, and evaluating the resulting technical trade-offs.

The specific objectives are:

- Analyze existing interoperability mechanisms between Rust and C++, including C ABI based integration, `cxx`, `bindgen`, `autocxx`, and `cbindgen`.
- Define explicit criteria for selecting Boost components suitable for migration, including safety relevance, isolation, portability, interoperability, performance relevance, and implementation complexity.
- Select and reimplement representative functionality from `Boost.Align` and `Boost.Endian` using Rust-native abstractions rather than performing a purely syntactic translation.

- Design interoperability boundaries with explicit ownership, lifetime, representation, and error-handling contracts.
- Measure functional correctness, selected performance characteristics, memory behaviour, and FFI overhead for the implemented case studies.
- Identify migration patterns that are generalizable beyond the two selected libraries and distinguish them from decisions that are specific to their APIs or compatibility requirements.
- Characterize the limitations of the evaluated approach, particularly at unsafe Rust and Rust/C++ FFI boundaries.

1.5 Research Contributions

The contributions of this dissertation are:

- A structured set of criteria for assessing Boost components as incremental Rust migration candidates.
- A comparative analysis of Rust/C++ interoperability mechanisms with explicit attention to compatibility, safety, portability, performance, ABI stability, and build complexity.
- A migration methodology that emphasizes semantic analysis, Rust-native redesign, explicit interoperability contracts, validation, and critical analysis of residual risks.
- Rust-native reimplementations of selected functionality from `Boost.Align` and `Boost.Endian`, demonstrating two different FFI strategies.
- An analysis of unsafe-code and FFI boundaries, including the safety properties that remain enforceable inside Rust and those that become external contracts.
- A quantitative evaluation of the selected implementations, including execution time, throughput, memory observations, correctness checks, and FFI overhead.

1.6 Dissertation Structure

The remainder of this dissertation is organized as follows:

- **Chapter 2 – State of the Art:** Presents the technological background, related work, Rust/C++ interoperability mechanisms, Boost architecture, and the criteria used to compare interoperability approaches.
- **Chapter 3 – Methodology and Solution Proposal:** Defines the component-selection criteria, migration-enabling rationale, and proposed architectural methodology.
- **Chapter 4 – Migration Strategy:** Describes the systematic migration process used to analyse, redesign, implement, integrate, and validate selected Boost functionality.

- **Chapter 5 – Selected Libraries:** Describes `Boost.Align` and `Boost.Endian`, their relevance to the study, and their selection as representative low-level case studies.
- **Chapter 6 – Implementation, Evaluation and Critical Analysis:** Presents the Rust implementations, benchmark results, unsafe-code analysis, FFI safety analysis, and generalizable migration patterns.
- **Chapter 7 – Conclusions and Future Work:** Summarizes the findings, answers the research questions, states the contributions and limitations, and identifies future work.

1.7 Privacy and ethics

In the context of this dissertation, which is focused in modernizing C++ based systems using Rust, ethical considerations were primarily centered on scientific integrity, software safety and intellectual property compliance.

The research relied on public open-source codebases, as Boost libraries. No personal data, user-identifiable information (PII), or private operational logs were collected, processed, or stored. Consequently, the study falls outside the scope of GDPR requirements regarding human subjects.

1.8 Use of AI Tools

For the preparation of this dissertation and supporting documentation, AI tools (specifically large language models) were used strictly for the following purposes:

- **Document writing evaluation and improvement**, including linguistic refinement, clarity enhancement, and reorganization of text originally written by the author.
- **Assistance in identifying potential sources and directions for literature review**, such as suggesting relevant topics, keywords, and areas of research to investigate. All cited references included in the dissertation were individually verified, read, and selected by the author.
- Structural feedback and general writing suggestions for non-technical sections.
- Code example generation to support the investigation of the state of the art and also to facilitate the understanding.

The tools used were Perplexity (latest version), for deep search especially for community based information, chat GPT (version GPT5.1) and Gemini (version 3), for document writing evaluation and improvement.

No AI-generated technical explanations, conceptual developments, design decisions, code, or analysis results were incorporated directly into the dissertation. All technical content is exclusively authored by the candidate.

The author takes full responsibility for the correctness, originality, and academic integrity of the work presented.

Chapter 2

State of the art

The integration of Rust into C++ environments, particularly Boost libraries, marks a significant evolution in software development. Rust's core strengths are memory safety and concurrency without garbage collection, addressing longstanding vulnerabilities in C++ systems. Recent studies on microcontroller applications show that Rust can have similar speed to C with lower energy use than interpreted languages. While performance evaluations on platforms, such as Raspberry Pi, indicates that Rust FFT implementations can match or exceed C in energy efficiency (Dashdamirli 2025; Rooney and Matthews 2023).

These characteristics are dominant in systems with strict predictable requirements, such as automotive Electronic Control Units (ECUs), high-frequency trading platforms, and real-time robotics. In these domains, the non-predictability introduced by garbage-collection "stop-the-world" pauses is not appropriate due to hard real-time latency constraints. Furthermore, resource-constrained embedded environments often cannot afford the runtime overhead associated with managed languages. Rust solves this problem through an ownership model and a strict borrow checker enforced at compile time. By tracking the lifetime and scope of every value during compilation, Rust guarantees memory safety and data-race freedom without requiring a runtime garbage collector. (Okawa and Yamaguchi 2024) This ensures that the resulting binary maintains the predictable performance and low resource footprint of C++, while eliminating entire classes of bugs such as null pointer de-referencing and use-after-free errors.

2.1 Rust

2.1.1 Overview

Rust is a systems programming language focused on three goals: safety, speed, and concurrency (Klabnik, Nichols, and Developers 2017).

It is designed to ensure memory safety and high performance without relying on a garbage collector, present in languages as Python or Java.

To not rely on garbage collection and to avoid common memory issues found in languages like C and C++, such as null pointer dereferencing, double frees, and segmentation faults, Rust was designed to be memory safe while offering comparable performance (Khan 2023).

Ownership and Memory Safety

Rust distinguishes itself through a rigorous ownership model, which manages memory through a strict set of rules enforced at compile time by the "borrow checker", without a garbage collector. (Blandy, Orendorff, and Tindall 2021)

The garbage collector, used by Python and Java, is an automatic memory management process that helps programs to run efficiently. Rust does not use it for memory management, instead it determines exactly when memory should be freed based on variable scope. (Blandy, Orendorff, and Tindall 2021)

Within a program run, every value has a single "owner" and when the owner goes out of scope the value is dropped. Ownership can be transferred (moved) but not implicitly copied for complex types, preventing double-free errors. (Blandy, Orendorff, and Tindall 2021)

I also possible to "borrow" values through references. The compiler ensures that references are always valid and a mutable references cannot exist while other references exists, preventing data races. (Khan 2023)

Scope and ownership code example:

```
fn main() {
// 1. SCOPE AND OWNERSHIP
{
    let s1 = String::from("hello"); // s1 owns the string
    // s1 is valid here
} // s1 goes out of scope here; memory is automatically freed (dropped)

// 2. MOVE SEMANTICS (Transferring Ownership)
let s2 = String::from("complex type");
let s3 = s2; // Ownership moved from s2 to s3

// println!("{}", s2); // Error! s2 is invalid. Prevents double-free errors.
println!("s3 owns the data: {}", s3);

// 3. BORROWING (References)
let mut s4 = String::from("data");

// Immutable borrow
let r1 = &s4;
let r2 = &s4; // Multiple immutable references are allowed
println!("Read only: {} and {}", r1, r2);
// r1 and r2 scopes end here (last usage)
```

```
23
24      // Mutable borrow
25      let r3 = &mut s4;
26      r3.push_str(" race"); // Modify the borrowed value
27      // let r4 = &s4; // Error! Cannot have immutable ref while mutable
        ref exists
28                        // This rule prevents data races.
29
30      println!("Modified data: {}", r3);
31  }
```

Tooling and ecosystem

An high quality tooling is a strong base of the Rust development environment. The integrated package manager **Cargo**, documentation generator **rustdoc** and built-in testing framework significantly enhance developer productivity. The Rust package registry, crates.io, is an important source of libraries for different applications.

Beside the growing usage of Rust, it still relies on interoperability with C/C++ ecosystem, specially for low-level applications (eg. hardware interactions, graphics, legacy protocols). Interfacing Rust with foreign code is possible using through the FFI. Usually, an FFI is functional and efficient but often requires careful design of abstractions, specially when crossing language boundaries with different memory management models.

Adoption and Industrial Applications

Rust has been adopted by several domains, as:

- Web browsers and rendering engines, as mozilla firefox (Mozilla 2026).
- Cloud infrastructures, as AWS firecracker (Amazon Web Services 2026).
- Operating systems and kernel modules, as in Linux kernel (The Linux Kernel Organization 2026).
- Blockchain clients and cryptographic systems (Kuo 2024).
- High performance networking and embedded systems (The Rust Project Developers 2026a).

2.2 C++ and the Boost Library

2.2.1 C++

C++ is currently the dominant programming language due to its extensive ecosystem. It has hardware specific libraries, mature compilers, profiling and debugging tools and frameworks for high-performance computing. Is also used for a large group of companies for many years (Stroustrup 2013). It leads to massive codebases, thousands of lines of code that make large rewrites infeasible (CppCon 2020).

Some common industries that usually use C++ are telecommunications, automotive, finance, aerospace, gaming, real-time control and large scale cloud systems (Stroustrup 2020).

The main challenges these systems have is their modernization. Modernizing them while ensuring the continuity, compatibility and performance is not easy. Due to this, languages that interoperate with C++ instead of replacing it are preferred.

The main strengths of C++ are:

- High performance (specially comparing with C) (Stroustrup 2020).
- Scope-based resource management through Resource Acquisition Is Initialization (RAII) (Stroustrup 2013).
- Support for control of hardware and memory (Stroustrup 2013).
- String abstraction mechanisms as templates, operator overloading and generic programming (Stroustrup 2013).
- Mature tools, compilers, profiling and debugging ecosystems (CppCon 2020).

The evolution of C++ ISO standard introduced modern features as smart pointers, concurrency primitives, modules and constant evaluation, for safety improvement and expressiveness (*ISO/IEC 14882:2020 Programming languages — C++* 2020).

C++ memory model and safety limitations

Beside all the advantages, C++ has memory and safety limitations. C++ is designed to give developers complete control over memory and resources lifetime. It leads to the following unsafe behaviors:

- Use after free and dangling pointers (*70% of Security Bugs Are Memory Safety Issues* 2020).
- Double frees (*70% of Security Bugs Are Memory Safety Issues* 2020).
- Buffer overflows and out of bound access (*70% of Security Bugs Are Memory Safety Issues* 2020).
- Iterator invalidation (Stroustrup 2013).

- Data races (Stroustrup 2013).
- Uninitialized memory reads (*70% of Security Bugs Are Memory Safety Issues* 2020).
- Reliance on undefined behaviors (Stroustrup 2013).

Industry data confirms that memory-unsafe languages (primarily C and C++) account for the majority of security vulnerabilities. Google reports that 70% of Chrome vulnerabilities stem from memory safety bugs (*Memory Safety in Android and Chrome* 2022), and Microsoft reports similar figures across Windows components (*70% of Security Bugs Are Memory Safety Issues* 2020). Although modern C++ attempts to mitigate issues through smart pointers and safer API design, legacy code and raw pointer usage remain widespread (CppCon 2020).

C++ concurrency and data races

C++ has a standardized model of memory since C++ 11. Through Standard C++ Library (STD), C++ provides access to threads (for multi processing and for parallel execution) and atomics that are a powerful mechanism for managing concurrent access to shared data in multithreaded applications. Also provides mutexes, locks and condition variables for safety memory access for multiple processes concurrently.

With all those features C++ provides a possibility of an high performance for parallel programs, but the correctness of C++ remains difficult. That is due C++ not being capable of statically prevent data races, C++ allows aliasing and mutation via multiple paths, exposes low level memory order operations and relies on the developer to ensure invariants (Stroustrup 2020).

At following example is possible to see two threads modifying the same resource, the global variable ***counter***. The modification is concurrently leading to a undefined behavior.

```
1   #include <iostream>
2   #include <thread>
3   #include <vector>
4
5   int counter = 0; // Shared resource
6
7   void increment() {
8       for (int i = 0; i < 1000000; ++i) {
9           // Data race occurs here: read-modify-write is NOT atomic
10          counter++;
11      }
12  }
13
14  int main() {
15      std::thread t1(increment);
16      std::thread t2(increment);
17
18      t1.join();
19      t2.join();
```

```
20
21          std::cout << "Final counter: " << counter << std::endl;
22          // Expected: 2000000
23          // Actual: Likely < 2000000 (e.g., 146377) due to race conditions
        [web:51]
24          return 0;
25  }
```

The incremental of ***counter***, **counter++**, is not atomic. Usually an incremental operation includes three steps: the load of the variable into a register, the value increment at the register and the value store back at the memory.

When this operation is not protected by locking (e.g. using threads) or atomic types (**std::atomic<int>**), both threads can load the same value from memory, increment it and store it, simultaneously. Since the initial value is the same, instead of expected increment by 2 it only will increment by 1.

2.2.2 Boost C++ Libraries

The Boost C++ Libraries are a comprehensive collection of peer-reviewed, portable, and open-source libraries that function as a testbed for the C++ Standard Library. Boost has heavily influenced modern C++ standards (C++11 through C++23), serving as the incubator for foundational components such as smart pointers, threading models, and regular expressions. Boost has a rigorous peer review process and commitment to portability made it a benchmark for high quality C++ library development. The capacity of early provision of production ready implementations of libraries, enabled developers to adopt it. (*Boost C++ Libraries Documentation* 2025)

2.2.3 The Boost Paradox: Barrier and Bridge

The Boost libraries have, historically, served as the proving ground for the C++ Standard Library. Boost provide high-quality, peer-reviewed, and largely header-only components that target portability and performance across platforms. (*Boost C++ Libraries Documentation* 2025) This ubiquity creates a unique dichotomy during migration, because the same abstractions that enabled modern C++ design patterns also amplify the complexity of introducing a Rust based ownership model into existing codebases.

- **Technical mismatch**: Boost utilizes advanced C++ features such as template metaprogramming, complex inheritance hierarchies, SFINAE (Substitution Failure Is Not An Error) that is a principal in C++ where an invalid substitution of template parameters is not an error in itself, and expression templates across libraries as `Boost.SmartPtr`, `Boost.Spirit`, and `Boost.Asio` (*Boost C++ Libraries Documentation* 2025).

These constructs don't have direct equivalents in Rust's generic system and trait bounds, particularly when combined with partial specialization and ADL-dependent overload sets. As a consequence, standard automated binding tools (such as `bindgen` (The rust-bindgen Project Developers 2026)) are ill-suited to interface with Boost's header-only, template-heavy design: the absence of concrete, non-templated symbols at the ABI level often forces developers to introduce intermediate C-compatible facades.

These facades are typically handwritten wrappers that manually erase templates into plain C types or opaque handles, reintroducing the same classes of human error (lifetime mismatches, missing error propagation, accidental copying) that Rust aims to eliminate in the first place.

This code uses a template, `async_wait` from the library boost::asio. It is not a compiled symbol in the library, is generated only when used. Rust is not able to link to it directly.

```
1        // async_engine.hpp
2        #include <boost/asio.hpp>
3        #include <iostream>
4
5        class AsyncEngine {
6        public:
7            AsyncEngine() : timer_(io_, boost::asio::chrono::
    milliseconds(500)) {}
8
9            // PROBLEM: This method uses a C++ Template for the
    handler (Lambda).
10           // Bindgen ignores this because it requires template
    instantiation.
11           // It relies on SFINAE to check if the handler has the
    right signature.
12           template <typename CompletionToken>
13           void start_task(CompletionToken&& token) {
14               timer_.async_wait(token);
15               io_.run();
16           }
17
18       private:
19           boost::asio::io_context io_;
20           boost::asio::steady_timer timer_;
21       };
22
```

- **Semantic opportunity**: Many Boost libraries were explicitly designed to encode disciplined resource management and concurrency patterns, that anticipate concerns later

formalized by Rust. Boost.SmartPtr library encapsulates ownership and shared lifetime through types, such as `boost::shared_ptr` and `boost::scoped_ptr`.

The library Boost.Thread provides higher-level threading primitives and synchronization constructs beyond the C++03 baseline (*Boost C++ Libraries Documentation* 2025).

Higher-level facilities, such as `Boost.Program_options` and `Boost.Asio`, encourage clear separation of configuration, I/O and business logic. It aligns well with Rust's preference for explicit data flow and minimal global state.

The migration opportunity lies in systematically leveraging these "vocabulary types" as a pivot layer: raw pointers and ad hoc ownership conventions at subsystem boundaries are first refactored into Boost abstractions, which can then be mechanically mapped to Rust equivalents (for example, `boost::shared_ptr<T>` → `std::sync::Arc<T>`, `boost::optional<T>` → `Option<T>`, or `boost::variant` → a Rust `enum`).

Treating Boost as a semantic bridge rather than a legacy liability allows the migration process to progressively tighten ownership and error-handling contracts on the C++ side before introducing Rust, thereby reducing the impedance mismatch at the FFI boundary.

A simulation of a shared configuration system based, it is usually used in ROS/robotics systems.

```
1        #include <vector>
2        #include <string>
3        #include <iostream>
4        #include <boost/variant.hpp>
5        #include <boost/shared_ptr.hpp>
6        #include <boost/make_shared.hpp>
7
8        // 1. Define the sum type: It's EITHER an int OR a string
9        typedef boost::variant<int, std::string> ConfigValue;
10
11       // 2. Define the container: A shared list of these values
12       typedef std::vector<ConfigValue> ConfigList;
13       typedef boost::shared_ptr<ConfigList> SharedConfig;
14
15       // Visitor to print values (Required to safely unpack the
    variant)
16       struct Printer : public boost::static_visitor<void> {
17           void operator()(int i) const { std::cout << "Speed: " <<
     i << std::endl; }
18           void operator()(const std::string& s) const { std::cout
    << "Mode: " << s << std::endl; }
19       };
20
21       int main() {
22           // 3. Create shared data
```

```
23              SharedConfig config = boost::make_shared<ConfigList>();
24              config->push_back(100);             // Add int
25              config->push_back("Autonomous");    // Add string
26
27              // 4. Share it (cheap copy, points to same data)
28              SharedConfig worker_ref = config;
29
30              // 5. Iterate and Visit (Type-safe access)
31              for (const auto& item : *worker_ref) {
32                  boost::apply_visitor(Printer(), item);
33              }
34
35              return 0;
36          }
37
38          // Rust equivalent using built-in equivalents (enum and
    match) instead of library templates
39          use std::sync::Arc;
40
41          // 1. Define the sum type (Enum)
42          enum ConfigValue {
43              Speed(i32),
44              Mode(String),
45          }
46
47          fn main() {
48              // 2. Create shared data (Vec wrapped in Arc)
49              // Note: Rust requires explicit variants (ConfigValue::
    Speed) unlike Boost
50              let config = Arc::new(vec![
51                  ConfigValue::Speed(100),
52                  ConfigValue::Mode(String::from("Autonomous")),
53              ]);
54
55              // 3. Share it (cheap clone, points to same data)
56              let worker_ref = config.clone();
57
58              // 4. Iterate and Match (Type-safe access)
59              for item in worker_ref.iter() {
60                  match item {
61                      ConfigValue::Speed(i) => println!("Speed: {}", i
    ),
62                      ConfigValue::Mode(s) => println!("Mode: {}", s),
63                  }
64              }
65          }
66
```

[language=C++]

2.3 Interoperability between Rust and C++

The transition from C++ to Rust represents a necessary evolution for high-assurance software, driven by the critical need to eliminate memory safety vulnerabilities characteristic of C++'s manual memory management. However, for systems deeply rooted in the C++ ecosystem, particularly those dependent on the Boost C++ Libraries, migration is not merely a syntactic translation but a complex architectural reconciliation. The core challenge lies in bridging two distinct paradigms without sacrificing the stability of massive legacy investments.

2.3.1 Undefined Behavior and Vulnerability Mitigation

A primary driver for this migration is the prevalence of undefined behavior in legacy C/C++ codebases, which often manifests as security vulnerabilities. A best-case historical study of open-source vulnerabilities indicates that approximately 58.2% of historical vulnerabilities in prominent C projects would have been virtually impossible in safe Rust. (Meneely et al. 2025)

Code vulnerabilities are formally listed at Common Weakness Enumeration (CWE). CWE was community developed and list common software and hardware weakness types.

Some critical vulnerabilities detected were:

- **Use-after-free (CWE-416)**: While Boost smart pointers manage object lifetimes, they cannot strictly enforce compile-time borrow checking, leaving potential for dangling references if raw pointers are extracted. Rust's borrow checker eliminates this entirely in safe code.
- **Buffer overflows (CWE-120/125)**: C++ containers (including `std::vector` and `boost::container`) rely on developer discipline to avoid out-of-bounds access. In contrast, Rust enforces bounds checking by default, preventing memory corruption.
- **Data races (CWE-362)**: Rust's `Send` and `Sync` traits ensure thread safety at the type level, preventing data races that are common in complex C++ multithreaded applications, even when using `boost::thread`.

However, automated translation tools often fail to fully capitalize on these safety guarantees. Transpilers like C2Rust may preserve the unsafe semantics of the original C code, necessitating further static analysis or refactoring to achieve true safety. (Hong 2023; Ling et al. 2022)

2.3.2 The Safety Air Gap in Hybrid Architectures

Hybrid architectures are structural design patterns that allow two languages to coexist in a single system, as systems with C++ and Rust. They share the same low-level control but

have different compilation models and safety guarantees. Due to it the architecture should focus on isolating the boundary to minimize friction.

Since big-bang rewrites, a software strategy where a system is rebuilt from scratch (rewriting a C++ system in Rust), are resource-prohibitive, organizations are forced into incremental migration.

This creates a prolonged hybrid state where Rust modules must coexist with legacy C++:

- **FFI vulnerabilities**: Rust's safety guarantees (ownership, borrow checking) strictly end at the Foreign Function Interface (FFI) boundary. Crossing this boundary destroys safety unless rigorously wrapped, creating a safety air gap. Dynamic analysis tools like Rustcheck have been proposed to detect memory safety issues specifically arising from unsafe Rust usage at these boundaries. (Xia, Wu, and Hua 2023)
- **Integration risks**: Passing complex Boost data structures across the FFI boundary introduces risks of unsafe pointer usage, exception mismatches, and memory leaks.
- **Concurrency conflicts**: Bridging Boost.Asio or Boost.Thread with Rust's concurrency model can lead to undefined behavior, specifically when distinct async runtimes or threading models attempt to interoperate without a unified strategy.

2.3.3 Systematic Porting vs. the Rewrite Trap

A lack of structured methodology often leads to the rewrite trap stalled development due to the sheer complexity of porting feature-rich systems.

- **Performance overhead**: FFI boundaries can introduce execution overhead, particularly when data conversion or marshalling is required. The magnitude of this overhead is workload- and interface-dependent, so it should be measured for the specific migration rather than assumed as a fixed percentage. In this dissertation, FFI cost is evaluated experimentally for the selected Boost.Align workload in Chapter 6.5.3.(The Rust Project Developers 2026b)
- **Missing methodology**: There is currently no standardized process for decomposing a Boost-heavy application into portable units. Developers need a systematic way to identify architectural seams where the cost of the FFI boundary is minimized and the safety value of Rust is maximized.

The central problem is not just the translation of code, but the architectural interoperability of two diverging memory models. The challenge is to define a strategy that stops treating Boost as legacy debt to be discarded and starts utilizing it as a stabilizing intermediate layer.

2.3.4 Foreign Function Interface (FFI)

A Foreign Function Interface (FFI) is a mechanism that enables a program written in one programming language (the host language) to call routines or make use of services written in another language (the guest language). FFIs are critical for system interoperability, allowing high-level languages to leverage legacy codebases, operating system APIs, and high-performance libraries often implemented in C or C++. (*C++ ABI Compatibility* 2025) In compiled languages like Rust, FFI is often handled through static declarations. The `extern` block defines function signatures that adhere to the C application binary interface (ABI) and Rust uses attributes such as `#[link(name = "...")]` together with `#[repr(C)]` on data structures to ensure layout compatibility and correct symbol resolution. (*Rust Documentation* 2025)

2.3.5 Interoperability tools

There are two main tools for interoperability between C/C++ and Rust: **cxx** and **autocxx**.

Cxx provides a safe mechanism for calling C++ code from Rust and Rust code from C++, not subject to undefined behaviors and memory safety risks often introduced when using **bindgen** or **cbindgen** to generate C-style bindings. Unfortunately, beside this, C++ code remains 100% unsafe.

AutoCxx is a tool for calling C++ from Rust in a heavily automated form. It includes all fluent safety from Cxx generating interfaces automatically from C++ headers using a variant of **bindgen**.

2.3.6 Evaluation Criteria for Interoperability Mechanisms

The interoperability mechanisms discussed above are not equivalent and should not be selected solely on the basis of whether they can technically connect Rust and C++. For an incremental migration of legacy Boost components, the mechanism must also satisfy constraints imposed by the existing C++ standard, the desired safety boundary, the target platforms, the build system, and the complexity of the exposed interface.

The following criteria are therefore used to compare the mechanisms considered in this dissertation:

- **C++ compatibility**: the minimum C++ language standard required by the interoperability mechanism and its ability to integrate with legacy C++03 code.
- **Safety**: the extent to which the mechanism can express interface contracts and reduce manually written FFI code, without assuming that the C++ implementation itself is memory-safe.
- **Portability**: dependencies on compiler versions, platforms, language standards, and generated artifacts.

- **Performance**: expected runtime and call-boundary overhead and the ability to expose low-level operations without unnecessary conversions.
- **ABI stability**: dependence on a C++ ABI versus a stable C-compatible boundary.
- **Template support**: ability to represent or wrap generic and template-heavy Boost interfaces.
- **Build complexity**: additional generators, headers, build-script integration, and maintenance requirements.
- **Auditability**: the ease with which the externally visible interface, ownership rules, and unsafe assumptions can be inspected and documented.

Table 2.1 summarizes the assessment. The values are qualitative suitability assessments for the migration scenario considered in this dissertation; they are not universal rankings of the technologies.

Criterion	**C ABI**	**cxx**	**bindgen**	**autocxx**	**cbindgen**
C++03 compatibility	High	Low	High	Low	High
Safety / interface checking	Medium	High	Medium	High	High
Portability	High	Medium	Medium	Medium	High
Performance transparency	High	High	High	Medium	High
ABI stability	High	Medium	Medium	Medium	High
Template-heavy C++ support	Low	Medium	High	High	N/A
Build complexity	Low	Medium	Medium	High	Low
Auditability	High	Medium	Medium	Medium	High

TABLE 2.1: Qualitative comparison of Rust/C++ interoperability mechanisms for the migration scenario

The comparison highlights an important methodological point: there is no single interoperability mechanism that is optimal for every Boost component. A small C-compatible interface is attractive when legacy C++03 compatibility and ABI stability dominate the requirements, whereas a higher-level bridge may provide more expressive interfaces when the C++ baseline permits it. This distinction is directly relevant to the two case studies: Boost.Align targets a C++03-compatible boundary, while the selected Boost.Endian integration uses `cxx` and therefore has a different compatibility envelope.

2.3.7 Boost migration for Rust

Currently there is no official migration of Boost from C++ to Rust. Both ecosystems, C++ and Rust, are designed within different paradigms. Boost is described as a peer reviewed

open source repository of C++ libraries. It is a massive, centralized collection that is used as testing ground for features eventually destined for STD.

Rust follows a different model. Instead of having a massive "Boost" crate, the community builds specialized, high-quality crates that are distributed through crates.io.

Boost, as previous reference, has an heavy use of Template Metaprogramming (TMP). It is not straightforward translation, the porting to Rust. That is due the fact of C++ and Rust handle generics differently. C++ templates are checked at compile time, if the code works. Rust generics are definition checked. Boost also relies on complex class hierarchies and inheritance. On other side, Rust uses composition, meaning a total architectural rewrite for Rust implementation of Boost. Boost also uses frequently smart pointers to manage complex object graphs and Rust uses ownership and borrowing that is a completely different approach. Those Rust rules are used to avoid the overhead of reference counting.

There is an entry at Crates.io for porting Boost libraries to Rust but there are no updates since 2021. At this moment has no source code (Zakarum 2023). There are also Rust-native alternatives to several Boost libraries available through the Rust ecosystem. The following table provides examples of such alternatives (*crates.io: The Rust Community's Crate Registry* 2025).

Boost Library	**Rust Equivalent**	**Notes**
Boost.Asio	`tokio`, `async-std`	Async I/O, networking, event loop
Boost.Beast	`hyper`, `reqwest`	HTTP client/server, WebSockets
Boost.Filesystem	`std::fs`, `std::path`	Native filesystem support in Rust stdlib
Boost.ProgramOptions	`clap`, `argh`, `pico-args`	Command-line parsing
Boost.Serialization	`serde`	General-purpose serialization framework
Boost.Regex	`regex`	Fast regular expression engine
Boost.Optional	`Option<T>`	Built-in language feature
Boost.Variant	`enum`, `enum_dispatch`	Tagged unions / sum types
Boost.Any	`dyn Any`	Type-erased storage
Boost.Graph	`petgraph`	Graph algorithms and data structures
Boost.Geometry	`geo`, `geo-types`	Computational geometry
Boost.SmartPtr	`Box`, `Rc`, `Arc`	Memory management primitives
Boost.Thread	`std::thread`, `crossbeam`	Threading and concurrency
Boost.Lockfree	`crossbeam`, `loom`	Lock-free data structures
Boost.Coroutine	`async/await` (built-in), `futures`	Cooperative concurrency

TABLE 2.2: Rust ecosystem crates that replace common Boost C++ libraries.

2.4 Case studies

After checking the IEEE paper library were not found specific case studies that directly migrate or benchmark Boost C++ libraries. The papers mainly focus on migrating C codebases or in specific FFI algorithms.

2.4.1 Concurrency Safety: Migrating Locking primitives

Legacy C software often uses `pthread_mutex_lock`/`pthread_mutex_unlock` to control concurrency. If C2Rust is used to migrate the C code directly to Rust, the resulting code still contains unsafe C style mutex operations, preventing Rust compiler from enforcing thread-safety guarantees (Immunant, Inc. 2025).

Was proposed, by Jaemin Hong, a solution to automatically migrate C programs to Rust with an high focus on concurrency. In order to solve C2Rust problems, was proposed a solution named Concrat. The solution Concrat consists on a aggregation of a C2Rust solution, a static analyzer and a Rust code transformer. The static analyzer performs the dataflow analysis of the code generated by the C2Rust to produce a summary about data and flow lock. The transformer replaces the lock API according to the previous summary.

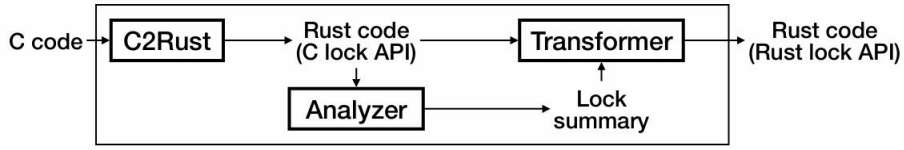


FIGURE 2.1: Concrat workflow (Hong 2023)

The tool achieved valid compilable Rust code in 29 of 46 cases collected from the github, proving that ownership-based concurrency can be automated even for legacy codebases (Hong 2023). From the 46 cases, 5 of them were also compilable but with a need of a manual fixes. The remain 12 cases were considered failures classified into three categories: functions conditionally acquiring locks, function pointers, and pointers to locks (Hong 2023).

Concrat focuses on C rather than C++, although its findings are applicable at C++ and also applicable to Boost C++ migration. Libraries as Boost.Thread and Boost.Asio have a strong dependency on manual synchronization, raw pointers and complex ownership patterns. The study has demonstrated that a simple translation of that code to Rust, relying on C2Rust or C++ bindings, would preserve the safety risks unless concurrency semantics are explicitly reconstructed. Concrat's static analysis offers a practical blueprint for the reconstruction providing a lock analysis.

This case study illustrates how automated, semantics-aware transformation can bridge the gap between C++'s flexible but unsafe concurrency model and Rust's strict, verifiable approach, enabling incremental and safer integration of Rust into existing C++ ecosystems.

2.4.2 "Just Use Rust": A Best-Case Historical Study of Open Source Vulnerabilities in C

In this paper the authors provide a historical security analysis of C language projects, based on the most common vulnerabilities of C and provide answers Rust based for those vulnerabilities. Those vulnerabilities were divide into two groups: the vulnerabilities Rust could prevent and the vulnerabilities Rust cannot prevent. Within this paper 68 prominent open source projects, as linux kernel and Curl, were analyzed in order to map the vulnerabilities.

The study identified a total of 4,507 vulnerabilities across these projects, 2,257 of which had specific Common Weakness Enumeration CWE designations. These were mapped to Rust's safety guarantees to determine prevention potential. The analysis reveals that a significant majority of historical security issues in C are memory safety errors that Rust effectively eliminates at compile time.

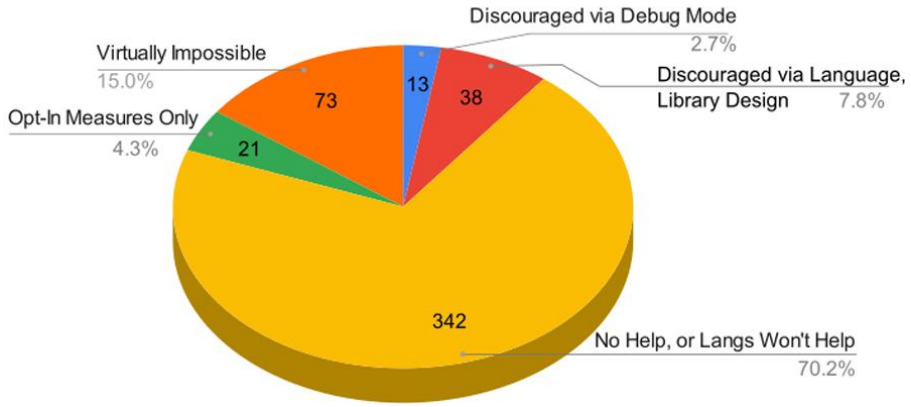


FIGURE 2.2: Rust Preventions based on CWE (Meneely et al. 2025)

As illustrated in Figure 2.2, the vulnerabilities are distributed across several categories of potential prevention through Rust. Approximately 15.0% fall into the "Virtually Impossible" category, indicating vulnerabilities that can be prevented by Rust's type system and ownership model. In contrast, approximately 70.2% fall into the "No Help, or Langs Won't Help" category, indicating vulnerabilities that cannot be prevented solely through the choice of programming language. The remaining vulnerabilities fall into intermediate categories where prevention depends on language features, libraries, compiler configurations, or development practices (Meneely et al. 2025).

Table 1 details the top 20 most common vulnerability types found in the subject projects. The data confirms that the two most frequent issues, NULL Pointer Dereference (CWE-476) and Use After Free (CWE-416), accounting for nearly 17% of all analyzed CVEs, are completely preventable in safe Rust (Meneely et al. 2025).

CWE ID	Vulnerability Name	# CVEs	# Proj	Rust Prevention
CWE-476	NULL Pointer Dereference	411	17	Virtually Impossible
CWE-416	Use After Free	343	17	Virtually Impossible
CWE-125	Out-of-bounds Read	145	14	Virtually Impossible
CWE-20	Improper Input Validation	105	17	No Help
CWE-190	Integer Overflow or Wraparound	100	12	Discouraged (Debug)
CWE-401	Memory Leak	96	6	Virtually Impossible
CWE-787	Out-of-bounds Write	92	15	Virtually Impossible
CWE-667	Improper Locking	86	4	Opt-In Measures
CWE-369	Divide By Zero	78	4	Virtually Impossible
CWE-617	Reachable Assertion	63	6	Discouraged (Debug)
CWE-362	Race Condition	62	11	Discouraged (Design)
CWE-119	Improper Buffer Restriction	62	14	Virtually Impossible
CWE-400	Uncontrolled Resource Consumption	59	15	No Help
CWE-122	Heap-based Buffer Overflow	53	12	Virtually Impossible
CWE-200	Exposure of Sensitive Information	45	11	No Help
CWE-120	Classic Buffer Overflow	45	9	Virtually Impossible
CWE-908	Use of Uninitialized Resource	35	5	Virtually Impossible
CWE-415	Double Free	34	8	Virtually Impossible
CWE-770	Resource Allocation without Limits	29	7	No Help
CWE-131	Incorrect Buffer Size Calculation	26	7	Virtually Impossible

TABLE 2.3: Top 20 Most Common Vulnerabilities and Rust Prevention Status (Meneely et al. 2025)

This data suggests that while Rust is not a "silver bullet" for all security issues (such as input validation or logic bugs), its adoption could have theoretically prevented the majority of the most severe historical vulnerabilities found in these critical infrastructure projects (Meneely et al. 2025).

Chapter 3

Methodology and Solution Proposal

The migration of critical software infrastructure from C++ to Rust is not a syntactic translation exercise but a fundamental architectural transformation. It requires rethinking ownership, concurrency, and interface boundaries in order to preserve functionality while improving safety guarantees.

As discussed in Chapter 2, the so-called “Boost Paradox” introduces a unique challenge: the very features that make Boost powerful—such as templates and metaprogramming—also hinder the applicability of automated interoperability tools.

This chapter defines a structured methodology to decompose Boost-based systems and proposes an architectural design that incrementally bridges the safety gap between C++ and Rust.

3.1 Component Selection Criteria

Historically, a “big bang” rewrite of large C++ codebases into Rust has proven prone to failure due to the difficulty of verifying feature parity, the disruption of ongoing development, and the elevated certification risk in safety-critical domains. Consequently, this research adopts a selective and incremental migration strategy.

Boost libraries are not considered uniformly; instead, they are evaluated and selected based on their relevance, isolation capability, and suitability for automotive software architectures.

3.1.1 Automotive-Oriented Selection Rationale

The automotive sector has increasingly adopted open-source and modular software platforms to address growing system complexity and multi-vendor integration. In this context, several automotive manufacturers and suppliers participate in Eclipse Software Defined Vehicle initiatives, including Eclipse Safety-Critical Open Research Environment (SCORE).

Eclipse SCORE is an open-source automotive platform initiative focused on a shared foundation for software-defined vehicles and high-performance ECUs, with safety and standards alignment forming part of its development context (Foundation 2025). These initiatives reflect

the realities of modern automotive development: large, heterogeneous C++ codebases, long product lifecycles, and strict constraints on timing, memory usage, and failure modes. Such characteristics render wholesale language replacement infeasible and reinforce the need for incremental, well-contained migration strategies.

The Boost libraries selected for migration are therefore evaluated according to the following automotive-oriented criteria:

- **Relevance to in-vehicle software**: Preference is given to libraries commonly used in middleware, communication, configuration management, and concurrency control, which are prevalent in automotive Electronic Control Units (ECUs) and service-oriented vehicle architectures.
- **Predictable behavior**: Libraries whose runtime behavior is predictable and suitable for real-time or near-real-time constraints are prioritized, aligning with automotive requirements for bounded latency and reproducible execution.
- **Safety and robustness**: Components that encode disciplined resource management or concurrency patterns (e.g., Boost.SmartPtr, Boost.Thread, Boost.Asio) are preferred, as they conceptually align with Rust's ownership and concurrency models.
- **Isolation capability**: Libraries that can be encapsulated behind stable, well-defined interfaces are favored, allowing them to serve as clear boundaries for Foreign Function Interface (FFI) integration.
- **Industrial and ecosystem relevance**: Libraries and migration patterns that are meaningful for open automotive software ecosystems, including Eclipse SCORE, are preferred because they provide a concrete architectural context for evaluating incremental interoperability and portability.

3.1.2 Operational Selection Criteria and Assessment

The criteria above are converted into an explicit assessment procedure so that component selection is traceable rather than based only on qualitative intuition. Each candidate is assessed against the following dimensions: automotive relevance, safety relevance, isolation capability, interoperability constraints, portability relevance, performance relevance, implementation complexity, and suitability for exposing reusable migration patterns.

Each dimension is rated on a five-point ordinal scale, where 1 indicates low relevance to the migration objective and 5 indicates high relevance. The scale is used as a structured decision aid rather than as an objective measurement of library quality. The assessment is documented before implementation so that the selection is not driven by the implementation results.

Criterion	Assessment question
Automotive relevance	Is the functionality relevant to low-level or embedded automotive software?
Safety relevance	Does the component expose memory, representation, ownership, or other safety-relevant concerns?
Isolation capability	Can the functionality be encapsulated behind a stable interface?
Interoperability relevance	Does the component expose meaningful Rust/C++ boundary-design challenges?
Portability relevance	Does the component involve platform, compiler, representation, or architecture concerns?
Performance relevance	Can meaningful performance characteristics be evaluated experimentally?
Implementation complexity	Is the component complex enough to exercise the methodology but bounded enough for a dissertation case study?
Pattern generalisability	Does the component expose migration decisions that can be separated from library-specific details?

TABLE 3.1: Operational criteria used for Boost component selection

Applying these criteria to the selected case studies gives the following qualitative assessment. The values are included to make the selection rationale explicit; they should not be interpreted as a universal ranking of Boost libraries.

Criterion	Align	Endian	Asio	Thread	SmartPtr	Optional
Automotive relevance	High	High	High	High	High	Medium
Safety relevance	High	High	High	High	High	Medium
Isolation capability	High	High	Medium	Medium	High	High
Interoperability relevance	High	High	High	High	Medium	Medium
Portability relevance	High	High	High	High	High	High
Performance relevance	High	Medium	High	High	Medium	Low
Implementation complexity	Medium	Medium	High	Medium	Low	Low
Pattern generalisability	High	High	High	High	High	Medium

TABLE 3.2: Qualitative assessment of candidate Boost libraries

The assessment was performed at the candidate-selection stage and is intended as a qualitative decision aid rather than as a benchmark, percentage-based score, or universal ranking of

Boost libraries. The additional candidates were selected because they expose migration concerns that complement the two implemented case studies. Boost.Asio and Boost.Thread introduce concurrency and asynchronous-execution interoperability concerns; Boost.SmartPtr directly exercises ownership and lifetime semantics; and Boost.Optional provides a smaller example of translating explicit optional-value semantics.

Boost.Align and Boost.Endian were retained because they expose complementary migration problems while remaining sufficiently bounded for controlled experimentation. Boost.Align concentrates on alignment, allocation, raw-pointer access, SIMD, and a C++03 compatibility boundary. Boost.Endian concentrates on binary representation, fixed-width values, generic abstractions, and a higher-level C++ interoperability mechanism. Together, these characteristics expose different points in the migration design space without claiming that they are the only or universally best candidates.

3.1.3 Boost.Align and Boost.Endian as Migration Enablers

The purpose of selecting Boost.Align and Boost.Endian is not to claim that they are representative of every complex Boost library. Instead, they are treated as **migration-enabling case studies**: each exposes a set of lower-level problems that must be solved before the methodology can be applied confidently to larger or more template-heavy components.

Boost.Align exposes patterns for representing alignment requirements, controlling ownership of allocated memory, isolating raw-pointer operations, defining opaque interfaces, and preserving interoperability with a C++03 consumer. Its implementation consequently provides a case study for translating memory-oriented C++ abstractions into Rust while keeping the unsafe surface explicit.

Boost.Endian exposes a different set of patterns. Its conversion, buffer, and arithmetic layers require explicit reasoning about binary representation, fixed-width types, generic abstractions, and C++ interoperability. The implementation uses these characteristics to investigate how semantic behaviour can be retained without reproducing the original C++ template hierarchy literally.

The resulting contribution is therefore the identification and evaluation of migration patterns rather than a claim that these two libraries solve the general Boost migration problem. The patterns that are potentially reusable include semantic rather than syntactic translation, explicit ownership contracts, isolation of unsafe operations, minimal FFI surfaces, and compatibility-driven selection of the interoperability mechanism. Decisions such as 32-byte alignment for the evaluated AVX workload or the use of a C ABI to preserve C++03 compatibility remain case-specific.

3.1.4 Deprioritized Components

Not all Boost libraries are suitable for migration or interoperability with Rust. Libraries that rely heavily on advanced template metaprogramming, domain-specific embedded languages, or extensive compile-time code generation are deprioritized due to their limited analyzability and poor suitability for FFI-based integration.

Similarly, libraries without a clear connection to the low-level concerns exercised by the automotive case-study context are not prioritized within the scope of this work. Eclipse SCORE is used as a motivating ecosystem rather than as evidence that the selected Boost libraries are already used by S-CORE.

3.1.5 Safety-Critical Constraints and ISO 26262 Alignment

ISO-26262 requires that software architectures for automotive safety systems support freedom from interference, predictable execution, and systematic fault avoidance. Programming languages and libraries used in such systems must minimize undefined behavior, provide explicit ownership semantics, and enable static verification wherever possible (ISO 2018).

C++'s permissive memory model and reliance on developer discipline pose challenges when targeting higher Automotive Safety Integrity Level (ASIL) levels. In practice, Boost libraries are frequently employed in automotive systems to impose structure and consistency on C++ development, particularly in resource management, concurrency, and asynchronous communication (Synopsys 2025).

The proposed methodology leverages this disciplined use of Boost as a preparatory step toward Rust integration. By constraining C++ systems to well-defined Boost abstractions, reliance on raw pointers, implicit ownership conventions, and ad hoc concurrency mechanisms is reduced. This aligns with ISO-26262 recommendations for limiting systematic faults through controlled language subsets, coding guidelines, and architectural enforcement (ISO 2018).

Rust components introduced under this methodology are designed to maximize the use of Rust's static ownership and type guarantees. Where unsafe operations are necessary, they are isolated, documented, and treated as explicit safety-relevant boundaries. The methodology supports safety analysis and auditability, but the dissertation does not claim certification or a complete safety case.

3.1.6 Boost Libraries Relevant to Eclipse SCORE

In Eclipse SCORE-style automotive architectures, Boost libraries are commonly used to support middleware services, configuration management, concurrency, and asynchronous I/O. Their selection is driven by the need to enforce structure and predictability in large-scale, safety-relevant C++ systems.

Table 3.3 summarizes Boost libraries commonly applicable to automotive platforms such as Eclipse SCORE and their corresponding Rust-native equivalents.

Boost Library	**Rust Equivalent**	**Automotive Relevance**
Boost.Asio	`tokio`, `async-std`	Asynchronous I/O and event handling for in-vehicle services and middleware; requires careful runtime integration in safety-critical contexts
Boost.Thread	`std::thread`, `crossbeam`	Thread management and synchronization for concurrent ECU software components
Boost.SmartPtr	`Box`, `Rc`, `Arc`	Explicit ownership and lifetime management; directly maps to Rust's ownership model
Boost.Optional	`Option<T>`	Explicit representation of optional values, reducing NULL pointer usage
Boost.Variant	`enum`	Type-safe sum types used in configuration, messaging, and state machines
Boost.Program_options	`clap`, `argh`	Configuration parsing for tooling and non-safety-critical control components
Boost.Filesystem	`std::fs`, `std::path`	File and path handling in diagnostic, logging, and update subsystems
Boost.Chrono	`std::time`	Time measurement and timeout handling with well-defined semantics

TABLE 3.3: Boost libraries relevant to Eclipse SCORE–style automotive systems and Rust equivalents

This mapping highlights that several Boost libraries already encode concepts such as explicit ownership, variant-based state, and structured concurrency— concepts that are native to Rust. As a result, the primary challenge of migration is architectural rather than conceptual, reinforcing the feasibility of incremental Rust adoption in safety-critical automotive ecosystems.

3.1.7 Comparative Analysis as a Migration Enabler

A central step in the proposed methodology is the comparison between Boost libraries used in automotive systems and their Rust-native equivalents. This comparison is not a simple replacement exercise, but an architectural analysis that exposes semantic gaps, safety assumptions, and ecosystem limitations.

By explicitly mapping Boost abstractions to Rust constructs, developers can:

- Identify which safety properties are already enforced at the type level
- Detect mismatches in ownership, error handling, or concurrency semantics

- Quantify the residual risk that remains after migration

This analysis also reveals areas where no suitable Rust equivalent exists for automotive-grade use. In such cases, the absence of a mature Rust library becomes an explicit design signal rather than an implementation obstacle.

3.1.8 From Library Gaps to New Rust Automotive Crates

The comparison between Boost and Rust ecosystems frequently uncovers functionality that is either missing or insufficiently mature in Rust for safety-critical automotive contexts.

Rather than forcing unsafe bindings or retaining C++ dependencies indefinitely, the proposed methodology treats these gaps as opportunities for new Rust library development. Such libraries are designed from the outset with:

- Explicit ownership and lifetime semantics
- Predictable execution models relevant to real-time analysis
- Minimal and auditable unsafe code
- Alignment with AUTOSAR Adaptive concepts

In this sense, Boost serves not only as a migration bridge, but also as a functional benchmark that informs the evolution of Rust's automotive ecosystem. The migration process thus becomes bidirectional: existing C++ designs guide Rust library development, while Rust's guarantees inform safer architectural patterns in C++.

3.1.9 ASIL-Oriented Migration Strategy

ISO-26262 distinguishes software components according to their ASIL, ranging from Quality Management (QM) to ASIL-D. Each level imposes increasing requirements on fault avoidance, fault detection, and freedom from interference. As a result, a uniform migration strategy is neither practical nor desirable across all ASIL levels.

The proposed methodology adopts a differentiated migration approach:

- **QM and ASIL-A components**: Tooling, diagnostics, logging, configuration services, and non-safety-critical middleware. These components are suitable early candidates for Rust adoption, as certification constraints are less restrictive.
- **ASIL-B and ASIL-C components**: Communication services, resource managers, and coordination logic. Rust integration at this level focuses on reducing the exposure to memory-safety defects while evaluating execution-time behaviour. Unsafe Rust is strictly isolated and subject to additional verification.
- **ASIL-D components**: Rust is introduced only where its guarantees provide demonstrable safety benefits. Components must be ownership-complete, free from dynamic

allocation where required, and analyzable under Worst-Case Execution Time (WCET) constraints. FFI boundaries are minimized and treated as safety-relevant interfaces subject to rigorous review.

This stratification aligns with ISO-26262 recommendations for mixed-criticality systems and supports freedom from interference between software elements of different ASIL levels.

3.1.10 Migration Workflow Overview

Figure 3.1 presents the proposed incremental migration workflow for integrating Rust into Boost-based automotive software systems. The workflow is designed to support safety-critical development under ISO-26262 constraints while preserving system continuity and certification traceability.

The process begins with a **comparative analysis** of existing Boost library usage and available Rust-native equivalents. This step establishes a semantic baseline by identifying ownership models, concurrency assumptions, error-handling mechanisms, and execution and timing properties already enforced in the C++ architecture.

Based on this analysis, an **architectural refactoring phase** follows, during which Boost abstractions are used to constrain C++ components into well-defined, analyzable interfaces. This step reduces semantic ambiguity, limits undefined behavior, and prepares the system for language boundary introduction without altering functional behavior.

Next, **FFI boundary definition** is performed. Stable and minimal interfaces are identified between C++ and Rust components, with explicit ownership transfer rules and data lifetime guarantees. These boundaries are treated as safety-relevant interfaces and are subject to additional review and verification.

The workflow then proceeds with **incremental Rust integration**, where Rust components are introduced selectively according to their assigned ASIL level. Initial integration targets QM and ASIL-A components, followed by progressively higher-criticality elements as confidence, tooling maturity, and safety argumentation evolve.

Throughout the process, **continuous verification and validation** are performed to ensure functional equivalence, timing behaviour, and freedom from interference. Feedback from each iteration informs subsequent migration steps, allowing the workflow to adapt to architectural constraints and certification requirements.

Overall, the workflow emphasizes early reduction of semantic ambiguity on the C++ side and controlled introduction of Rust components, providing a more explicit interoperability boundary. Any claim of automotive safety compliance remains dependent on the complete system, development process, and applicable safety case.

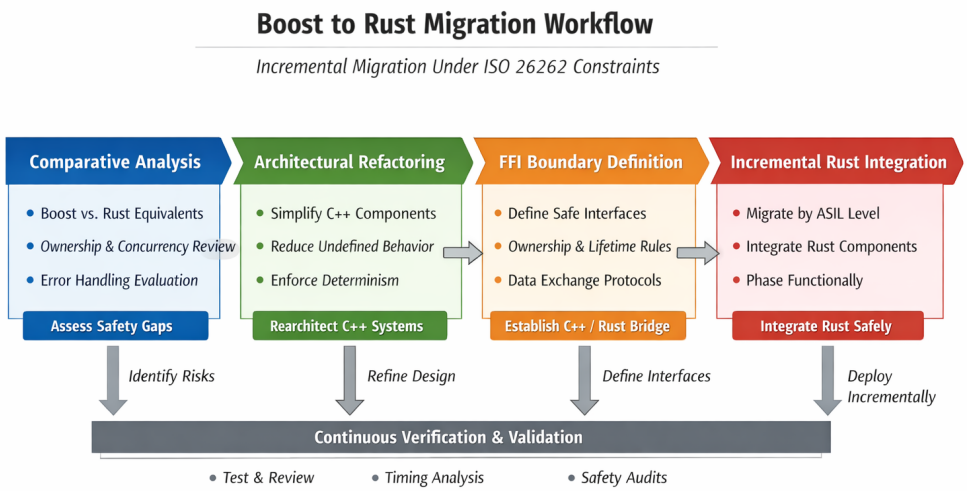


FIGURE 3.1: Incremental migration workflow for Boost-based automotive systems integrating Rust under ISO-26262 constraints

3.2 Research Design and Evaluation Plan

The proposed methodology is evaluated through two bounded case studies, `Boost.Align` and `Boost.Endian`. The case-study design is intended to provide evidence about the migration patterns identified by the methodology without implying that the results are general measurements for all Boost libraries or all automotive software.

3.2.1 Research Design

Each case study follows the same high-level sequence: analysis of the original Boost functionality, definition of equivalent Rust abstractions, implementation of the selected functionality, integration through an appropriate FFI mechanism, functional validation, and performance evaluation. This common structure makes it possible to distinguish migration decisions that arise from the general methodology from those that are specific to a library.

Boost.Align is used to evaluate low-level memory alignment, allocation, pointer handling, SIMD-related operations, and compatibility with a C++03 consumer. Boost.Endian is used to evaluate binary representation, fixed-width values, generic abstractions, and C++/Rust interoperability through a higher-level interface. The two cases therefore exercise complementary migration concerns while remaining sufficiently bounded for controlled experimentation.

The automotive relevance is motivated by the requirements of modern software-defined vehicle platforms. Eclipse S-CORE is an open-source automotive software platform aimed at

providing a shared foundation for high-performance ECUs and safety-critical vehicle software, with emphasis on interoperability, portability, and automotive standards (Foundation 2025). In this context, the selected libraries are not presented as components that must be migrated in S-CORE itself. Instead, S-CORE is used as a motivating architectural context in which incremental migration of low-level C++ infrastructure can be relevant. The case studies consequently act as migration enablers: they expose alignment, representation, ownership, and FFI patterns that can later inform work on larger and more template-intensive components.

3.2.2 Development Workflow

The implementation follows four methodological phases: analysis; design and implementation; interoperability; and validation and evaluation. The first phase establishes the semantic baseline of the Boost implementation and identifies relevant interfaces, ownership assumptions, dependencies, and observable behaviour. The second phase defines Rust-native abstractions and implements the selected functionality without requiring a literal translation of the C++ structure. The interoperability phase exposes only the interface required by the existing C++ consumer. The final phase validates functional behaviour and evaluates the measured characteristics of the resulting implementation.

This workflow deliberately separates the conceptual methodology from the implementation of each case study. The detailed migration procedure used within these phases is described in Chapter 4, while the implementation and experimental results are presented in Chapter 6.

3.2.3 Validation and Evaluation Strategy

Validation combines functional tests, interoperability checks, and benchmark measurements. Functional validation verifies that the selected Rust functionality produces the expected results for the evaluated inputs. Interoperability validation verifies that the exposed interfaces can be consumed from the corresponding C++ environment. Performance evaluation measures execution time and throughput for the workloads defined for each case study.

The evaluation is intentionally interpreted as experimental evidence for the selected workloads. It does not establish worst-case execution time (WCET), jitter bounds, schedulability, hard real-time guarantees, or a complete ISO 26262 safety case. Likewise, process-level memory measurements are treated as observations of the benchmark process, not as proof of lower memory consumption in general.

3.2.4 Experimental Configuration

The experiments use Rust 1.93 and Boost 1.91.0 as the implementation and reference environments, respectively. Boost.Align is evaluated against a C++03-compatible interface, while Boost.Endian uses the C++ interoperability mechanism selected for its implementation. The

benchmark configurations use release builds and repeated measurements; the reported results are presented with the exact workload parameters in Chapter 6.

The evaluation focuses on three evidence categories: functional equivalence, measured performance, and interoperability/safety-boundary characteristics. These categories are sufficient to assess the feasibility of the proposed migration patterns while avoiding claims that would require broader system-level experiments.

3.2.5 Evaluation Scope and Limitations

The selected case studies represent only a subset of Boost functionality. The results are therefore used to evaluate the migration approach and identify practical trade-offs rather than to generalize performance or safety properties to the complete Boost ecosystem. In particular, the evaluation does not include complete vehicle-level integration, long-duration operational testing, WCET analysis, multicore scheduling analysis, formal verification, or certification assessment. These limitations are considered explicitly when interpreting the results and the research questions in Chapter 7.

Chapter 4

Migration Strategy

This chapter defines a systematic strategy for migrating selected Boost libraries to Rust. The strategy establishes a common methodology that is applicable to each of the selected libraries and provides a structured approach for evaluating the resulting implementations.

The primary objective of the proposed strategy is to reproduce the relevant functionality of the selected Boost libraries in Rust while preserving their expected behavior and enabling their integration with existing C++ codebases. In addition, the strategy considers the requirements and constraints associated with the potential use of the migrated libraries in safety-critical automotive software architectures.

The migration strategy is divided into six main steps. These steps operationalise the methodology introduced in Chapter 3 and provide a common procedure for both case studies:

1. **Understanding the library's functionality and its dependencies.**

   The first step consists of analyzing the selected Boost library's source code, documentation and relevant technical literature, to understand its purpose, functionality, interfaces and dependencies.

   This analysis identifies the relevant classes,functions, data structures, algorithms, platform-specific behavior and interactions with other libraries. Particular attention is given to aspects that affect the observable behavior of the library and that may influence its migration to Rust.

   A comprehensive understanding of these aspects is essential to ensure that the Rust implementation reproduces the relevant semantics of the original library and can be subsequently be integrated into existing C++ codebases.

2. **Define Rust equivalents for the library's classes, functions, and data structures.**

   Based on the analysis of the library's original implementation, the equivalent Rust abstractions are defined for the library's classes, functions, data structures and interfaces.

   The Rust design does not necessarily replicate the structure of the original C++ implementation. Where appropriate, the design may differ in order to take advantage of

Rust's ownership model, type system, error handling mechanisms and idiomatic programming practices, while preserving the required behavior of the original library.

3. **Implement the library's functionality in Rust, ensuring that it adheres to Rust's ownership and borrowing properties and best practices.**

   The library's functionality is reimplemented in Rust. During this stage, particular attention is given to the Rust's ownership and borrowing model, ownership and borrowing properties, error handling and idiomatic design principles.

   The implementation also considers the requirements and constraints of intended target environment. Where the use of `unsafe` Rust is necessary, such usage should be minimized and restricted to well-defined boundaries, particularly when interacting with external code or platform-specific functionality.

4. **Create a C++ wrapper for the Rust implementation, allowing it to be used in existing C++ codebases.**

   To enable integration with existing C++ codebases, an interoperability layer (C++ wrapper) is developed around the Rust implementation.

   The interoperability layer provides an interface suitable for use by the existing C++ applications while isolating the underlying Rust implementation from the consuming code. Particular attention is given to the representation of data, ownership and object lifetime, error handling, memory management and the interaction between the C++ and Rust components.

5. **Test the Rust implementation and the C++ wrapper to ensure that they function correctly and meet performance requirements.**

   The Rust implementation and its C++ wrapper are tested to verify functional correctness, behavioral equivalence with the original Boost library where applicable, interoperability and performance.

   The original Boost implementation serves as a reference when behavioral compatibility is required. Equivalent inputs and relevant execution scenarios are used to identify discrepancies between the original and migrated implementations.

   Performance evaluation is also performed to determine whether the Rust implementation and the interoperability layer introduce unacceptable overhead compared with the original implementation.

6. **Document the migration process, including any challenges encountered and solutions implemented, to provide guidance for future migrations of other Boost libraries to Rust.**

   The final step consists of documenting the migration process, including design decisions, encountered challenges, solutions, testing results and lessons learned.

This documentation provides a reference for future migrations to other Boost libraries and contributes to the definition of a repeatable migration methodology.

Among these steps, two are particularly critical for the success of the migration strategy. The first is the analysis of the library's functionality and dependencies, since an incomplete understanding of the original implementation can result in incorrect assumptions about its behavior or dependencies. The second is the **implementation of the functionality in Rust**, as this stage requires translating the original C++ abstractions into Rust while preserving the required semantics and taking advantage of Rust's ownership and borrowing properties and type system. These two stages therefore have a significant influence on the correctness and overall success of the migration.

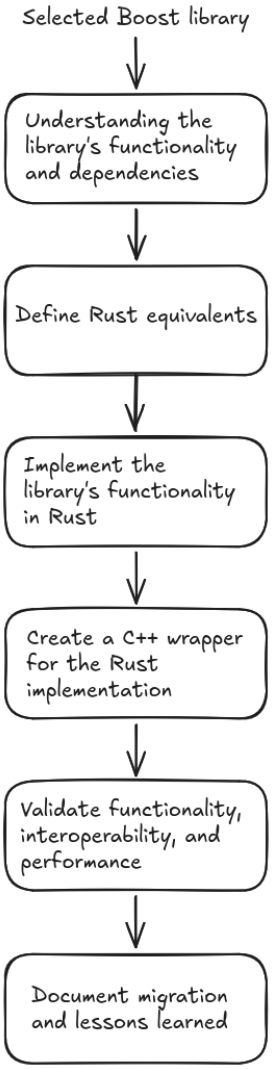


FIGURE 4.1: Migration strategy

The six steps described above can be grouped into four main phases, as follows:

1. **Analysis**
   (a) Understanding the library's functionality and dependencies.
   (b) Defining the migration scope and requirements.
2. **Design and Implementation**
   (a) Defining the Rust equivalents of the relevant C++ abstractions.
   (b) Implementing the required functionality in Rust.
3. **Interoperability**
   (a) Developing the interoperability layer between Rust and C++.
4. **Validation and Evaluation**
   (a) Testing functional correctness, behavioral equivalence, interoperability, and performance.
   (b) Documenting the results, limitations, and lessons learned.

The phases are illustrated in Figure 4.2.

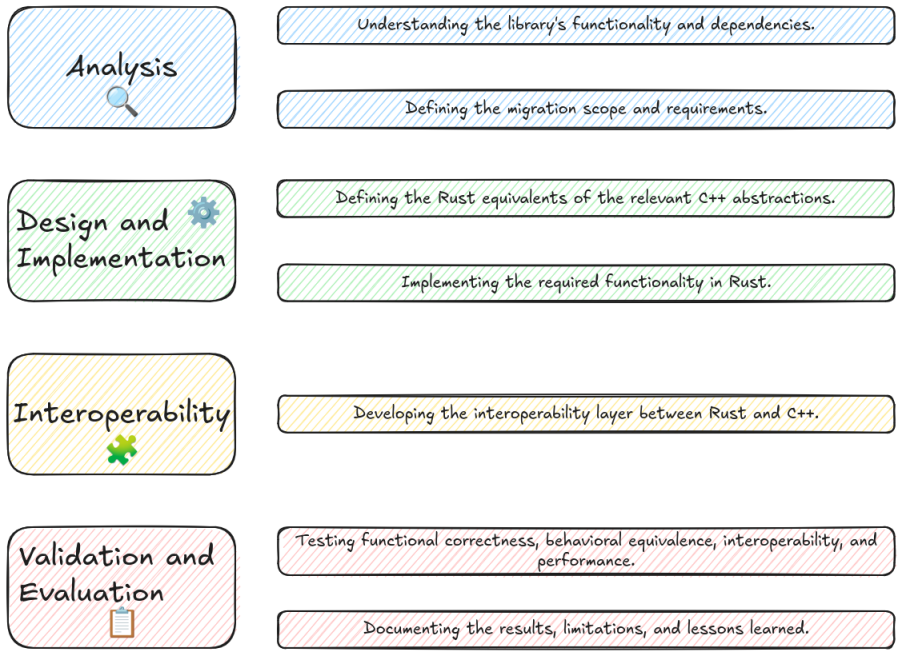


FIGURE 4.2: Migration phases

The migration strategy aims to satisfy the following objectives:

- Preserve the relevant behavior of the original Boost library;
- Provide equivalent functionality through a Rust implementation;
- Maintain interoperability with existing C++ applications;
- Leverage Rust's memory and type safety guarantees;
- Maintain acceptable performance compared with the original implementation;
- Minimize and isolate the use of `unsafe` Rust, particularly at interoperability boundaries;
- Consider the requirements and constraints of the intended automotive environment.

The success of a migration is therefore evaluated according to the extent to which these objectives are satisfied. The specific application of the proposed strategy, including the analysis, implementation, interoperability and evaluation of each selected Boost library, is presented in the following chapter.

Chapter 5

Selected Libraries

This chapter presents the selected Boost libraries for reimplementation in Rust. The selection is based on several criteria, including their relevance to automotive software architectures, implementation characteristics, complexity and the potential benefits that a Rust implementation may provide in terms of memory safety, reliability and performance.

The selected libraries are: `Boost.Align` and `Boost.Endian`. Both libraries provide relatively low-level functionality that is relevant to systems programming and embedded software, while also exposing functionality that interacts closely with memory representation and processor architecture. These characteristics make them suitable candidates for evaluating the feasibility of reproducing existing C++ functionality using idiomatic Rust.

For each selected library, this chapter provides an overview of its purpose and functionality, describes its main components and discusses its relevance to automotive software development. The chapter also identifies the main challenges associated with reimplementing the library in Rust, including differences between the C++ and Rust memory models, language abstractions, portability requirements and interoperability with existing C++ codebases.

5.1 Boost.Align

5.1.1 Overview

The `Boost.Align` library provides functions, classes, templates, traits and macros, for controlling, inspecting and diagnosing memory alignment. Its primary purpose is to provide portable alignment-related functionality across different compilers, standard library implementations and language standards.

Memory alignment refers to the requirement that an object is stored at a memory address satisfying a particular alignment constraint. For example, a type with an alignment requirement of 8 bytes must generally be placed at an address divisible by 8. Alignment requirements are determined by the target architecture and by the type being stored. Correct alignment is

important because processors may impose restrictions on the addresses from which particular types can be accessed. Even on architectures that support unaligned accesses, aligned accesses can provide more predictable and efficient memory operations.

Alignment is particularly relevant to low-level and performance-sensitive software. Incorrectly aligned objects may result in inefficient memory accesses or, on architectures with stricter alignment requirements, hardware faults. In C++, violating the alignment requirements of an object can also result in undefined behavior. Consequently, explicit control over memory alignment can be important when implementing data structures, custom allocators, memory pools, serialization mechanisms, and other low-level components.

Before C++ 11, the language provided limited standardized facilities for expressing and manipulating alignment requirements. Boost.Align addressed this limitation by providing portable alignment utilities that could be used in C++03 code and across implementations with incomplete or inconsistent support for newer alignment-related facilities.

C++11 introduced standardized support for specifying alignment requirements through language and library facilities. In particular, the language introduced alignment specifications through `alignas`, while the standard library introduced facilities such as `std::align`. However, compiler and standard library support was not always complete or consistent at the time Boost.Align was developed. The library therefore also provided portable implementations and compatibility facilities for environments in which the corresponding standard functionality was unavailable or unreliable.

5.1.2 Functionality

Boost.Align provides functionality for both dynamic memory allocation and pointer alignment. Its facilities can broadly be divided into allocation, pointer alignment, alignment inspection, and alignment diagnostics.

For dynamic memory allocation, the library provides:

- `aligned_alloc(alignment, size)`, which allocates a block of memory satisfying a specified alignment requirement;
- `aligned_free(ptr)`, which releases memory previously allocated using the corresponding aligned allocation mechanism.

These functions are useful when the default allocation provided by the runtime or standard library does not guarantee the alignment required by an application or data structure.

The library also provides allocator abstractions, including:

- `aligned_allocator<T>`.;
- `aligned_allocator_adaptor<Allocator>`.;
-

- `aligned_delete<T>`..

The allocator abstractions allow alignment requirements to be integrated with C++ allocation mechanisms and standard containers. This is particularly useful for containers whose elements must satisfy an alignment requirement greater than the default alignment provided by the allocator.

Another important part of Boost.Align concerns pointer alignment. The library provides an implementation of functionality equivalent to `std::align`, allowing a pointer within an available memory region to be adjusted to satisfy a requested alignment while preserving the required amount of available storage.

This functionality is relevant when implementing custom memory-management mechanisms. For example, an application may reserve a larger memory region and subsequently divide it into several aligned regions for different objects. Explicit pointer alignment allows such regions to be placed at addresses compatible with the alignment requirements of the objects they contain.

The library additionally provides facilities for querying and inspecting alignment requirements. These capabilities allow software to determine the alignment associated with a type or address and to verify whether an address satisfies a particular alignment constraint. Such facilities are useful when implementing generic low-level components where the alignment requirements cannot be assumed statically.

5.1.3 Relevance to automotive software

Memory alignment is particularly relevant to automotive software because many automotive systems operate under strict constraints regarding performance, memory consumption, predictability, and hardware compatibility. Software components may interact directly with hardware, communication buffers, memory-mapped data, DMA regions, or specialized data structures whose layout and alignment must be carefully controlled.

For example, an automotive application may process large collections of sensor data or communication frames using structures with specific alignment requirements. Correct alignment can contribute to efficient memory accesses and may be necessary to satisfy the requirements of a particular processor or hardware interface.

Alignment can also become important when software is designed to run across multiple processor architectures. Automotive platforms may use different CPU architectures and compiler toolchains, each with potentially different alignment characteristics. A portable abstraction for alignment can therefore reduce the amount of platform-specific code required by a software component.

Furthermore, automotive software frequently contains legacy C++ codebases that were developed before the widespread availability of modern C++ standards. Such systems may

continue to rely on Boost libraries to provide functionality that was historically unavailable or insufficiently portable in the C++ language and standard library.

Although many of the capabilities historically provided by Boost.Align are now available through modern C++ language and standard library facilities, Boost.Align remains relevant when maintaining or migrating legacy systems. Reimplementing these facilities in Rust therefore provides an opportunity to investigate how low-level memory-management functionality originally designed for C++ can be expressed using Rust's stronger type system and ownership model.

5.1.4 Challenges of reimplementation in Rust

Reimplementing Boost.Align in Rust presents several challenges. The most significant is the interaction between explicit memory alignment and Rust's safety guarantees.

Rust provides alignment information as part of its type system and guarantees that references to properly typed values satisfy the corresponding alignment requirements. However, operations involving raw pointers, manually allocated memory, and custom allocation strategies may require the use of `unsafe` code. Consequently, an implementation of Boost.Align-like functionality must carefully isolate unsafe operations and ensure that the safety invariants expected by the Rust type system are preserved.

Dynamic aligned allocation is another important consideration. Rust's allocation APIs allow an allocation to be described through a `Layout`, which contains both the size and alignment of the requested memory region. This provides a natural abstraction for implementing functionality similar to Boost.Align's aligned allocation mechanisms. Nevertheless, the implementation must correctly handle invalid layouts, allocation failures, deallocation, and the relationship between an allocation's size and alignment.

Pointer alignment presents an additional challenge because it requires reasoning about raw memory addresses and available storage. A Rust implementation must ensure that pointer arithmetic does not violate the language's safety requirements and that any resulting pointer is only converted into a reference when all required invariants are satisfied.

Allocator integration also differs significantly between C++ and Rust. Boost.Align integrates with the C++ allocator model and standard containers, whereas Rust uses the `GlobalAlloc` and allocator-related abstractions available in the language ecosystem. Therefore, a direct translation of the C++ interfaces would not necessarily result in an idiomatic Rust implementation. Instead, the reimplementation should preserve the observable semantics of the original library while adapting its interface to Rust's ownership, borrowing, and allocation abstractions.

Finally, portability must be considered. The original Boost library was designed to compensate for differences between compilers and standard library implementations. A Rust implementation should therefore be evaluated across the relevant target architectures and

toolchains to determine whether its behavior is consistent and whether platform-specific assumptions have been introduced.

5.1.5 Rationale for selection

Boost.Align was selected because it represents a low-level library whose functionality is closely related to memory management and processor architecture. It therefore provides a useful case study for evaluating the migration of C++ systems-level functionality to Rust.

The library is also particularly suitable for examining the boundary between safe and unsafe Rust. While many high-level uses of aligned data can be expressed safely through Rust's type system, explicit allocation and pointer manipulation require lower-level primitives. This makes Boost.Align useful for assessing whether Rust can provide a safer abstraction around functionality that traditionally requires careful manual reasoning in C++.

In addition, Boost.Align provides a relatively compact and well-defined API while still containing several distinct concepts, including alignment calculation, pointer manipulation, dynamic allocation, and allocator integration. This provides sufficient implementation complexity to evaluate the migration methodology without making the selected library unnecessarily large.

Although alignment support has substantially improved in modern C++ and much of the functionality provided by Boost.Align is now represented by standard C++ facilities, the library remains relevant when dealing with older C++ codebases and portability requirements. Its reimplementation in Rust can therefore demonstrate how existing low-level C++ infrastructure can be reproduced while taking advantage of Rust's memory-safety guarantees and modern abstractions.

The original Boost.Align documentation and source code were used as the primary references for the analysis and subsequent reimplementation:

- `https://github.com/boostorg/align/blob/boost-1.91.0/doc/align.qbk`
- `https://www.boost.org/library/latest/align/`
- `https://www.boost.org/doc/libs/latest/doc/html/align.html`

5.2 Selection Summary and Migration-Enabling Patterns

The selection of `Boost.Align` and `Boost.Endian` is intended to expose complementary migration patterns. The libraries are not treated as complete representatives of the Boost ecosystem; instead, they provide bounded case studies through which the methodology can be exercised on low-level functionality with different interoperability requirements.

Library	Primary migration challenge	Migration pattern exposed
`Boost.Align`	Alignment, allocation, pointer access, SIMD, and C++03 compatibility	Safe abstraction around low-level operations; explicit unsafe isolation; minimal C ABI; ownership-aware opaque interface
`Boost.Endian`	Binary representation, fixed-width values, generic buffer/arithmetic abstractions, and C++ interoperability	Explicit representation types; semantic adaptation of generic C++ abstractions; higher-level FFI boundary

TABLE 5.1: Migration-enabling patterns exposed by the selected libraries

Several decisions are therefore generalizable across migrations: analysing observable semantics before implementation, defining a bounded migration scope, redesigning rather than mechanically translating abstractions, minimizing unsafe code, making FFI contracts explicit, and validating the migrated behaviour against the reference implementation. Other decisions remain case-specific. For example, the 32-byte alignment used by the Align workload, the AVX implementation, the C++03 compatibility constraint, and the use of `cxx` for the selected Endian bridge are properties of the corresponding case studies rather than universal migration rules.

The value of these case studies as migration enablers is consequently methodological. They demonstrate concrete solutions to recurring classes of problems—memory representation, ownership, unsafe operations, ABI boundaries, and semantic adaptation—that are likely to reappear when larger Boost components are considered, while avoiding the unsupported claim that successful migration of these two libraries establishes feasibility for every template-heavy Boost component.

5.3 Boost.Endian

5.3.1 Overview

The `Boost.Endian` library provides facilities for manipulating the byte order, or endianness, of integer and user-defined types. Endianness defines the order in which the individual bytes of a multi-byte value are represented in the memory. The two predominant byte-ordering schemes are little-endian, where the least significant byte is stored first, and big endian, where the most significant byte is stored first.

Endianness is generally transparent to application level software when the data remains within a single system. However, it becomes relevant when binary data is exchanged between systems, stored in files, transmitted over networks, or interpreted by software running

on different processor architectures. In such cases, the representation of a value in memory may differ from the representation expected by the receiving system.

As example, considering a 16-bit value such as `0x0102` is represented as the byte sequence `0x01 0x02` on a big-endian system and as `0x02 0x01` on a little-endian system. If the byte representation is interpreted without considering its endianness, the received application may obtain a different numerical value, leading to incorrect behavior or data corruption. Consequently, explicitly defining and handling byte order is an important requirement for portable binary data formats and communication protocols.

Boost.Endian was developed to provide a portable and consistent abstraction for these operations. The library supports binary data portability independently of the native endianness of the host platform and provides a common interface across different C++ environments. It also addresses a secondary use case involving data size optimization by providing integer representations with sizes that are not directly available as standard C++ integer types, such as 24-bit, 40-bit and 48-bit integers. These types can be useful when implementing compact binary data formats or communication protocols.

The library is header only and in its current version, requires C++11. It can make use of compiler provided byte swapping intrinsics when available, allowing implementations to take advantage of architecture specific instructions while retaining a portable interface.

5.3.2 Functionality

Boost.Endian provides three complementary approaches for working with endian dependent data: endian conversion functions, endian buffer types and endian arithmetic types. Each approach is designed for a different balance between explicitness, ease of use, performance and control over when conversions take place.

Endian conversion functions

The conversion function approach uses the standard C++ integer types to store values and explicitly converts their representation when required. The library provides both mutating and non-mutating conversion operations, including unconditional and conditional variants.

For example, an application can convert a value from big endian to the native byte order before performing calculations and convert it back before writing the value to an external representation. This approach makes the conversion points explicit and is particularly appropriate when an application already has an established data model based on the native C++ types.

The conversion functions also provide a relatively direct abstraction over underlying processor byte swapping operations. When compiler intrinsics are available, Boost.Endian can use them to generate efficient machine code.

Endian buffer types

Endian buffer types store values in a representation with a specified byte order. Unlike ordinary native integer types, the representation maintained by the buffer does not change depending on the host architecture.

The library provides buffer types for a range of integer sizes, including 8-bit, 16-bit, 24-bit, 32-bit, 40-bit, 48-bit, 56-bit and 64-bit integers. Both aligned and unaligned representations are available, with unaligned representations being particularly useful when dealing with tightly packed binary formats.

An important characteristic of buffer types is that the conversions are explicit. A value must be explicitly loaded from or stored into the endian representation. This prevents hidden conversions and gives the programmer precise control over when byte order transformation occurs.

This property is useful for binary protocols and file formats where the in memory representation must remain stable regardless of the architecture on which the software is executed.

Endian arithmetic types

Endian arithmetic types combine a fixed byte-order representation with an interface similar to the built-in C++ arithmetic types. They support arithmetic operations while automatically performing the required conversions between the stored representation and the native representation.

This approach simplifies application code because explicit conversions do not have to be performed for every arithmetic operation. For example, an application can maintain a structure containing big-endian integer fields and operate on those fields using normal arithmetic expressions.

The convenience of implicit conversion comes with a potential performance trade-off. Repeated operations on endian arithmetic types may result in repeated conversions, particularly inside computationally intensive loops. Boost.Endian therefore also documents patterns for moving conversions outside performance-critical loops when necessary.

5.3.3 Binary data portability

One of the primary motivations for Boost.Endian is binary data portability. Binary formats frequently specify the exact byte ordering and size of their fields. Unlike textual representations, binary formats cannot rely on the receiving system to interpret values according to its native representation.

This is particularly important for network protocols, file formats, serialization systems, and inter-process communication mechanisms. A data structure containing several integer fields may have to be transmitted from a little-endian processor to another system whose native

representation is different. Explicitly representing the required byte order ensures that the same logical value is reconstructed at the destination.

Boost.Endian therefore separates the representation of externally defined binary data from the native representation of the host machine. This allows the application to use a consistent representation for data exchanged with external systems while remaining independent of the processor's native byte order.

The library also supports unaligned integer representations. This is useful for binary formats where fields are packed without the padding that would normally be introduced by C++ object alignment. Boost.Endian specifically identifies unaligned 24-, 40-, 48-, and 56-bit representations as useful when the exact size of the external representation matters.

5.3.4 Relevance to automotive software

Endianness is particularly relevant to automotive software because modern automotive systems consist of heterogeneous hardware and software components that communicate using precisely defined binary data formats.

Automotive systems commonly exchange data between electronic control units, sensors, gateways, communication interfaces, and higher-level computing platforms. These components may use different processors, operating systems, compilers, and software stacks. Consequently, communication interfaces cannot generally rely on the native representation of a particular processor.

The problem becomes particularly relevant when software processes binary communication frames or serialized data structures. A protocol may define a field as a 16-bit or 32-bit integer with a specific byte ordering, independently of the architecture executing the software. The implementation must therefore interpret the field according to the protocol definition rather than according to the host architecture.

This characteristic makes endian handling relevant to automotive communication stacks, serialization libraries, diagnostic protocols, and other components operating close to hardware and communication interfaces.

The relevance of these concerns is also reflected in modern automotive software platforms. The Eclipse S-CORE project, for example, is an open-source initiative focused on a software-defined vehicle platform and provides an example of the increasing role of modern software architectures and cross-platform technologies in automotive systems.

For an automotive software architecture, using explicit endian-aware representations can therefore help establish a clear boundary between the internal representation of application data and the representation required by an external communication or storage format.

5.3.5 Memory representation and portability

Boost.Endian is also relevant to portability because it avoids assuming that the host architecture uses the same byte order as the external data source.

A naïve implementation may interpret a binary structure directly through a native C++ structure. Such an approach can introduce portability problems because the resulting representation can depend on the processor's endianness, alignment requirements, padding rules, and compiler behavior.

For example, a structure containing several native integer fields cannot necessarily be treated as a portable representation of a network or file format simply by writing its memory contents directly to a file or communication channel. The representation of the fields may differ between platforms, and additional padding may be introduced by the compiler.

Endian-aware types provide a mechanism for explicitly representing the byte order required by the external format. This allows the application to distinguish between the representation used for communication and the representation used internally for computation.

This distinction is particularly valuable in safety- and reliability-oriented systems because it makes assumptions about binary representation explicit rather than relying on implicit properties of the target architecture.

5.3.6 Performance considerations

Byte-order conversion is generally a low-level operation that can be implemented efficiently using processor instructions. Boost.Endian takes advantage of compiler intrinsics when available, including byte-swap operations provided by GCC, Clang, and Microsoft Visual C++.

The performance characteristics of the different Boost.Endian approaches depend on how frequently conversions occur. When conversion functions are used, the programmer can explicitly control when a value is converted. This can be advantageous when a value is used repeatedly because the conversion can be performed once before a computationally intensive operation and the result converted back only when necessary.

Endian arithmetic types provide greater convenience but may perform implicit conversions during arithmetic operations. Boost.Endian's documentation demonstrates that, particularly in repeated computations, explicitly controlling the location of conversions can avoid unnecessary byte swapping.

This distinction is relevant for automotive applications because software may simultaneously have strict requirements for portability and computational efficiency. Communication-oriented code may prioritize explicit representation and correctness, while performance-critical processing may require minimizing repeated conversions.

5.3.7 Challenges of reimplementation in Rust

Reimplementing Boost.Endian in Rust presents several challenges, although Rust provides several primitives that are well suited to the problem.

The first challenge is representing values with an explicitly defined byte order while maintaining Rust's type and memory-safety guarantees. Rust's standard library provides methods such as `from_be`, `from_le`, `to_be`, and `to_le`, as well as byte-oriented conversion facilities. These provide a strong foundation for implementing endian-aware abstractions, but they do not directly reproduce the complete Boost.Endian API.

A second challenge is the implementation of endian-aware buffer types. Such types need to preserve a specific byte representation independently of the host architecture. A Rust implementation can use byte arrays as the underlying representation, which has the advantage of making the representation explicit and avoiding direct dependence on native integer layout.

Unaligned representations introduce another challenge. Rust provides strong guarantees around references and alignment, meaning that interpreting an arbitrary byte sequence directly as an integer reference may require unsafe operations. A safe implementation should therefore prefer byte-based representations and explicit conversion operations where possible, isolating any required unsafe code behind a well-defined abstraction.

A further challenge is reproducing Boost.Endian's arithmetic types. Boost.Endian allows endian-aware values to participate directly in arithmetic expressions, while Rust's operator traits provide a different abstraction mechanism. Implementing equivalent behavior requires implementing traits such as `Add`, `Sub`, `Mul`, and their assignment variants where appropriate.

However, a direct translation of the C++ API would not necessarily produce an idiomatic Rust library. Rust's ownership model, trait system, explicit conversion mechanisms, and emphasis on compile-time safety provide opportunities to design a safer interface rather than reproducing the C++ interface exactly.

Another important consideration is floating-point support. Endianness of floating-point representations cannot always be treated as a simple integer byte swap because floating-point representation and byte ordering involve additional assumptions. Boost.Endian therefore places restrictions on some floating-point conversion operations. A Rust reimplementation must preserve these semantic limitations rather than assuming that arbitrary floating-point values can safely be transformed by reversing their bytes.

5.3.8 Rationale for selection

Boost.Endian was selected because it represents a particularly relevant example of low-level, cross-platform functionality that is commonly required when implementing systems software.

Unlike functionality that is primarily concerned with convenience abstractions, endian handling directly affects the representation of data exchanged between software components.

This makes the library particularly relevant to automotive architectures, where multiple software and hardware components must exchange binary data according to well-defined protocols.

The library also provides an interesting migration target because it contains several distinct implementation strategies: conversion functions, buffer types, and arithmetic types. These approaches expose different trade-offs between explicitness, ease of use, memory representation, and performance. Reimplementing them in Rust provides an opportunity to evaluate how these design choices can be expressed using Rust's type system and trait-based abstractions.

The support for both aligned and unaligned representations is another reason for its selection. It introduces a direct connection to low-level memory representation and requires careful consideration of alignment, raw byte access, and safe abstractions. This complements the selection of `Boost.Align`, which focuses explicitly on memory alignment.

The combination of `Boost.Align` and `Boost.Endian` is therefore particularly useful for the migration study. While Boost.Align focuses on the physical placement and alignment of objects in memory, Boost.Endian focuses on the representation and interpretation of multi-byte values. Together, they cover two closely related aspects of low-level systems programming: how data is positioned in memory and how that data is represented across system boundaries.

Furthermore, Boost.Endian is sufficiently mature and well-defined to provide a meaningful basis for comparing the original C++ implementation with a Rust implementation. The library's documented performance considerations, multiple API approaches, and portability requirements provide several dimensions against which the Rust implementation can be evaluated.

The primary references used for the analysis are:

- `https://www.boost.org/library/latest/endian/`
- `https://www.boost.org/doc/libs/latest/libs/endian/doc/html/endian.html`
- `https://github.com/eclipse-score`

Chapter 6

Implementation, Evaluation and Critical Analysis

6.1 Implementation Overview

This chapter presents the implementation of the Boost to Rust migration described in the previous chapters. The implementation focuses on two selected Boost libraries, namely `Boost.Align` and `Boost.Endian`, which were selected due to their low level nature, relevance to systems programming and suitable for evaluating the migration methodology.

The implementation translates the relevant functionality of the original C++ libraries into Rust, while avoiding a direct syntactic translation of the original source code. Instead, the Rust implementation uses Rust's ownership model, type system and standard library facilities where these provide suitable abstractions for the original C++ functionality.

The implementation is evaluated with respect to four main objectives: functional equivalence, memory safety, interoperability with existing C++ software and suitability for automotive systems software.

The implementation follows the migration strategy described in Chapter 4. For each library, the original functionality is first mapped to Rust abstractions, followed by the implementation of the selected functionality and its associated tests. Where interoperability is required, an appropriate interoperability interface is introduced around the Rust implementation. The specific FFI mechanism is selected according to the compatibility requirements of each library.

6.2 Implementation Environment

The implementation was developed in the `boostinrust` repository, which provides the experimental environment for evaluating the migration of selected Boost libraries to Rust. The implementation focuses on `Boost.Align` and `Boost.Endian`, selected as representative examples of low level systems functionality involving memory alignment and binary data representation.

The Rust implementation was developed using Rust version 1.93. The development environment includes the Rust compiler (`rustc`), Cargo and the LLVM/Clang based tooling provided with the Rust toolchain. In particular, the environment contains the Rust compiler and the associated `rust-clang`, `rust-lld`, `rust-llvm-dwp`, and `rustdoc` components. The use of the LLVM/Clang toolchain is relevant to the low-level nature of the selected libraries and provides the compiler infrastructure required for building and inspecting the resulting Rust implementation.

The original Boost implementations are used as the reference implementations for the migration. This allows the Rust implementation to be evaluated against the functionality and observable behavior of the original C++ corresponding libraries. The approach follows the migration strategy described in Chapter 4, in which the functionality and dependencies of the original library are first analyzed before equivalent Rust abstractions are defined and implemented. The resulting Rust implementation is then tested and evaluated against the original C++ library to ensure functional equivalence, memory safety, interoperability and suitability for automotive systems software.

The implementation does not attempt to reproduce the internal structure of the C++ implementation directly. Instead, Rust specific mechanisms are used where they provide a more appropriate representation of the original functionality. This includes Rust's ownership and borrowing model, type system and standard library facilities for memory layout and byte order operations. This distinction is particularly relevant for the selected libraries. Boost.Align requires operations involving memory alignment, allocation and pointer manipulation, while Boost.Endian requires explicit control over the byte representation and byte order of stored values.

The implementation is structured to support functional verification through tests. The behavior of the Rust implementation is evaluated against the expected behavior of the corresponding Boost functionality, with particular attention given to boundary conditions and low-level memory operations. This follows the development plan established in the methodology, where functional equivalence is verified through unit testing and interface-level validation.

A specific concern of the implementation environment is the use of `unsafe` Rust. Although Rust provides alignment and memory-safety guarantees through its type system, reproducing some low-level Boost functionality requires operations involving raw pointers or manually allocated memory. such operations are therefore isolated and minimized in the implementation. This follows the migration strategy's requirement to preserve Rust's safety guarantees while restricting unsafe operations to well-defined boundaries.

Table 6.1 summarizes the main tools and versions used during the implementation.

Component	Version / Configuration
Rust	1.93
Rust compiler	`rustc 1.93`
Documentation	`rustdoc 1.93`
Compiler infrastructure	LLVM / Clang
Linker	`rust-lld`
Reference implementation	Boost 1.91.0

TABLE 6.1: Implementation environment

The resulting environment provides a controlled basis for implementing and evaluating the selected libraries. It combines the original Boost implementations used as behavioral references with a contemporary Rust toolchain, allowing the migration to be evaluated while preserving the low-level characteristics relevant to systems programming.

6.3 Boost.Align Reimplementation

The reimplementation of Boost.Align focuses on the memory-alignment functionality required by the selected use case. Rather than reproducing the complete Boost.Align API, the implementation provides a Rust-native abstraction for creating and manipulating 32-byte-aligned data and exposes the relevant functionality through a C-compatible interface.

The implementation is located in the align component of the project. Its central abstraction is AlignedBuffer, which stores floating-point data in an alignment-aware representation suitable for SIMD processing. The implementation additionally provides an `aligned_type!` macro for defining aligned Rust types and an AVX-specific processing function for operating on aligned buffers.

This approach follows the migration strategy described in Chapter 4: the objective is not to reproduce the original C++ implementation internally, but to preserve the relevant observable behaviour while using Rust's type system, ownership model and memory-management mechanisms. In particular, the implementation uses Rust's `#[repr(align(...))]` attribute and Vec allocation rather than implementing a separate low-level aligned allocator.

6.3.1 Scope and API mapping

The Boost.Align library provides several facilities related to memory alignment, including aligned allocation and deallocation, pointer alignment, alignment inspection and aligned allocators. For the implementation developed in this project, only a subset of this functionality was required.

The Rust implementation therefore maps the relevant alignment requirements to Rust language and standard-library mechanisms rather than attempting to reproduce the complete Boost.Align API.

Boost.Align concept	Rust implementation	Purpose
Aligned storage	`#[repr(align(32))]`	Establishes a compile-time alignment requirement
Aligned type	`Aligned<T>`	Provides a generic 32-byte-aligned wrapper
User-defined aligned type	`aligned_type!`	Generates types with a configurable alignment
Aligned memory buffer	`AlignedBuffer`	Stores aligned `f32` data
Aligned allocation	`Vec<F32x8>`	Uses Rust's allocator with the alignment of `F32x8`
Pointer access	`as_ptr()` / `as_mut_ptr()`	Provides raw-pointer access for SIMD and FFI
Alignment-dependent processing	`double_f32_avx()`	Performs AVX operations on aligned memory
C++ interoperability	`ffi` module	Exposes the buffer and SIMD operations through C ABI functions

TABLE 6.2: Mapping of Boost.Align concepts to the Rust implementation

This represents a deliberate reduction of the original API surface. In particular, the current implementation does not provide direct Rust equivalents of all Boost.Align functions such as align_up, align_down, is_aligned, aligned_alloc, aligned_free, or aligned_allocator. Instead, alignment is established structurally through the type definition of the stored elements.

This distinction is important because the objective of the implementation is to investigate whether the relevant alignment guarantees can be expressed more directly using Rust's type system rather than reproducing Boost's low-level allocation interface.

6.3.2 FFI Design and C ABI Selection

The Boost.Align reimplementation uses a C-compatible ABI to expose the Rust implementation to existing C++ software. The selection of this approach is primarily driven by the C++ compatibility requirements of the original Boost.Align library. Boost.Align supports C++03, which is therefore considered the baseline C++ standard for the interoperability requirements of this migration.

Rust–C++ interoperability frameworks such as `cxx` were therefore not selected for the Boost.Align interoperability layer. In particular, the CXX build integration requires a C++11 compilation

environment, which is incompatible with the C++03 baseline that must be preserved for the Boost.Align use case. Using CXX would consequently introduce a newer C++ standard requirement and prevent the resulting Rust replacement from being integrated into C++03-based code while maintaining the compatibility target of the original library.

A C ABI provides a suitable alternative because the interface can be consumed from C++03 without requiring modern C++ language constructs at the interoperability boundary. The Boost.Align FFI exposes a deliberately small set of C-compatible functions operating on primitive values, sizes, buffers, and opaque pointers. No C++ classes, templates, exceptions, or standard-library types are required by the interface.

The C ABI also separates the C++ compatibility requirements from the internal Rust implementation. The Rust implementation can use modern Rust features, including its ownership model, type system and standard-library abstractions, while the externally visible interface remains compatible with the C++03 environment. This allows the migrated implementation to preserve the integration requirements of the original Boost.Align library without constraining the internal Rust implementation to the capabilities of C++03.

The decision can therefore be summarized as follows:

- **C++03 compatibility is a requirement** because it represents the baseline C++ standard supported by the Boost.Align use case.
- **`cxx` and similar Rust–C++ FFI approaches were not selected** because their C++ requirements are incompatible with this C++03 baseline.
- **The C ABI was selected** because it provides the required interoperability boundary without introducing a dependency on a newer C++ standard.
- **Rust-specific implementation details remain internal**, allowing the Rust implementation to use idiomatic Rust abstractions while exposing only a C-compatible interface to the C++ application.

This decision is specific to the Boost.Align migration. The Boost.Endian reimplementation uses `cxx` because its interoperability requirements and compatibility constraints differ. Consequently, the two libraries demonstrate different approaches to Rust–C++ interoperability while allowing the migration methodology to be evaluated under different integration requirements.

The resulting Boost.Align architecture is:

```
C++03 application
|
| C-compatible ABI
v
Boost.Align FFI layer
|
```

```
v
Rust AlignedBuffer
|
v
Rust allocation and SIMD implementation
```

The C ABI therefore acts as the compatibility boundary that allows the Rust implementation to replace the Boost.Align functionality while retaining compatibility with the C++03 environment targeted by the original library.

6.3.3 Alignment representation

The central design decision in the Rust implementation is to represent alignment as a type-level property.

The implementation defines:

```
#[repr(align(32))]
pub struct Aligned<T>(pub T);
```

The implementation defines:

```
#[repr(align(32))]
pub struct Aligned<T>(pub T);
```

The `#[repr(align(32))]` attribute requires instances of `Aligned<T>` to have an alignment of at least 32 bytes. The wrapper implements `Deref` and `DerefMut`, allowing access to the contained value without requiring callers to manipulate the wrapper explicitly.

A second mechanism is provided through the `aligned_type!` macro, which can be invoked as `aligned_type!(AlignedF32x32, f32, 32)`.

The macro generates a structure equivalent to:

```
#[repr(align($align))]
pub struct $name(pub $inner_type);
```

This provides a convenient mechanism for defining aligned types while keeping the alignment requirement explicit at the type definition.

The implementation moves an important part of th alignment responsibility from runtime allocation logic to the Rust type system. Instead of allocating and arbitrary block of memory and

subsequently determine the returned pointer satisfies an alignment requirement, the type of the element carries the required alignment.

The buffer implementation applies the same principle through the internal `F32x8` type:

```
#[repr(align(32))]
struct F32x8([f32; 8]);
```

The `F32x8` type represents eight `f32` values, corresponding to a 256-bit AVX register. The `#[repr(align(32))]` attribute requires instances of `F32x8` to have an alignment of at least 32 bytes. Consequently, a `Vec<F32x8>` allocation is required to satisfy this alignment requirement.

The `F32x8` type represents eight `f32` values, corresponding to a 256-bit AVX register. Since the type has an alignment requirement of 32 bytes, a `Vec<F32x8>` allocation is required to satisfy this alignment.

Consequently, the alignment requirement is established by the type of the allocation rather than by manually adjusting an address returned from a general-purpose allocation.

The logical length of the buffer is maintained separately:

```
pub struct AlignedBuffer {
data: Vec<F32x8>,
len: usize,
}
```

Here, `data` contains the physically allocated and aligned storage, while `len` records the number of logical `f32` elements stored in the buffer.

The constructor explicitly restricts the buffer size to multiples of eight:

```
assert!(size % 8 == 0,
"AlignedBuffer size must be a multiple of 8");
```

This restriction ensures that the logical buffer consists exclusively of complete `F32x8` blocks. It also establishes the condition required by the AVX implementation, which processes eight `f32` values per iteration.

The implementation enforces two constraints:

1. the underlying storage has an alignment of 32 bytes; and
2. the number of logical `f32` elements is a multiple of eight.

Together, these properties ensure that the buffer can be safely used with aligned AVX load and store operations, with each operation processing a complete group of eight `f32` values.

This approach establishes the alignment requirement through the type of the allocation itself, rather than by manually adjusting an address returned by a general-purpose allocation. The alignment constraint is therefore expressed as part of the type definition, allowing the Rust allocator and collection types to preserve the required alignment without requiring explicit pointer manipulation by the caller.

6.3.4 Aligned Memory Allocation

Different from original Boost.Align interface, the Rust implementation does not explicitly expose a general purpose `aligned_alloc`/ `aligned_free` pair. Instead, aligned storage is obtained indirectly through the Rust `Vec` abstraction.

The relevant type definition is:

```
#[repr(align(32))]
struct F32x8([f32; 8]);
```

The aligned elements are then stored in the `AlignedBuffer` structure:

```
pub struct AlignedBuffer {
data: Vec<F32x8>,
len: usize,
}
```

The constructor creates the vector as follows:

```
data: (0..chunks)
.map(|_| F32x8([value; 8]))
.collect(),
```

The important property of this approach is that the allocator operates on `F32x8` elements rather than directly on `f32` elements. The alignment requirement of the allocated storage is therefore derived from the alignment requirement of `F32x8`. This allows the alignment constraint to be expressed through the type rather than being supplied explicitly as an argument to the allocation operation.

This represents a significant difference from the Boost.Align implementation. Boost.Align exposes allocation as an explicit operation in which the caller specifies both the requested size and alignment. The Rust implementation, in contrast, encodes the alignment requirement into

the type being allocated and relies on the standard `Vec` abstraction to manage the resulting storage.

The ownership model is also substantially different. The `AlignedBuffer` owns its `Vec<F32x8>`, and the allocated storage is automatically released when the `AlignedBuffer` is dropped. Consequently, normal Rust usage does not require an explicit deallocation function or require the caller to pair an allocation with a corresponding `aligned_free` operation.

This ownership-based approach eliminates an important class of manual memory-management operations present in the C++ interface. Resource lifetime is instead tied to the lifetime of the owning Rust value, with deallocation performed automatically according to Rust's ownership and drop semantics.

An explicit destruction function is retained only for the C compatible interface, where manual lifetime management is required to interoperate with external code. This distinction allows the native Rust API to remain idiomatic while still providing the necessary lifecycle operations for the interoperability layer described later.

6.3.5 Pointer Alignment

Although the storage is represented using an aligned type, the implementation provides raw-pointer access through the following methods:

```
pub fn as_ptr(&self) -> *const f32 {
self.data.as_ptr() as *const f32
}

pub fn as_mut_ptr(&mut self) -> *mut f32 {
self.data.as_mut_ptr() as *mut f32
}
```

This functionality is required because the SIMD implementation operates directly on raw pointers. The AVX operation is defined as follows:

```
#[target_feature(enable = "avx")]
pub unsafe fn double_f32_avx(ptr: *mut f32, len: usize)
```

The implementation processes eight elements per iteration:

```
for i in (0..len).step_by(8) {
let v = _mm256_load_ps(ptr.add(i));
let result = _mm256_add_ps(v, v);
_mm256_store_ps(ptr.add(i), result);
```

```
5 }
```

The `_mm256_load_ps` and `_mm256_store_ps` operations require appropriately aligned memory. Consequently, the caller of `double_f32_avx` must satisfy three requirements:

- the pointer must be aligned to 32 bytes;
- the length must be a multiple of eight; and
- the complete memory range must be valid and exclusively accessible for the duration of the operation.

The alignment guarantee originates from the `F32x8` type, while the multiple-of-eight constraint is established by the construction of `AlignedBuffer`.

The SIMD function nevertheless retains an `unsafe` boundary because Rust cannot statically verify all safety conditions associated with an arbitrary raw pointer supplied by the caller. In particular, pointer validity, the size of the accessible memory region, exclusive access for mutable operations and the processor feature required by the function cannot all be inferred from the raw-pointer interface.

For the buffer-based FFI function, the implementation additionally performs a runtime alignment check before passing the pointer to the AVX implementation:

```
1 assert_eq!(
2 slice.as_ptr() as usize % 32,
3 0,
4 "buffer pointer is not 32-byte aligned: {:p}",
5 slice.as_ptr()
6 );
```

This assertion provides a runtime sanity check at the FFI boundary, where data may originate from code outside the guarantees of the native Rust API.

The resulting design combines static guarantees provided by the Rust type system with runtime validation at the unsafe and FFI boundaries. The aligned storage type establishes the required memory alignment, while the buffer abstraction constrains its logical length. Runtime checks are then used to validate assumptions that cannot be expressed or verified statically when crossing the raw-pointer and FFI interfaces.

6.3.6 Allocator Support

The original Boost.Align implementation provides dedicated aligned allocator abstractions, including `aligned_allocator` and `aligned_allocator_adaptor`. The Rust implementation

does not reproduce these classes directly. Instead, allocation is delegated to Rust's standard `Vec` abstraction.

The relevant allocation is a `Vec<F32x8>`, where `F32x8` has a 32-byte alignment requirement:

```
1 Vec<F32x8>
```

This means that the alignment requirement of the allocated storage is derived from the alignment of the type being stored rather than being provided as a separate parameter to the allocation operation.

This design provides two advantages for the selected use case. First, it avoids introducing a separate custom allocator abstraction merely to reproduce the C++ allocator interface. Second, ownership remains explicit: `AlignedBuffer` owns the vector and therefore owns the memory represented by it.

The lifetime of the allocation is consequently tied to the lifetime of the `AlignedBuffer` object. When the object is dropped, its `Vec<F32x8>` is also dropped and its allocation is released automatically by Rust's memory-management mechanisms.

The implementation therefore replaces the explicit C++ allocation and deallocation relationship with Rust's ownership model:

```
Boost.Align:
aligned_alloc()
|
raw pointer
|
aligned_free()

Rust:
AlignedBuffer
|
Vec<F32x8>
|
automatic Drop
```

This is not a one-to-one API translation. Instead, it represents a semantic mapping from explicit manual allocation to an ownership-based allocation model. For the selected AVX use case, this provides the required alignment guarantee without exposing the allocation mechanism directly to the caller.

6.3.7 Unsafe Code Isolation

One of the implementation objectives was to restrict unsafe operations to the smallest practical part of the implementation. Most of `AlignedBuffer` is implemented using safe Rust, with construction, ownership, length management, and destruction handled through standard Rust abstractions.

Unsafe operations are concentrated in three areas.

The first is the conversion from the internally stored `F32x8` representation to an `f32` slice:

```
pub fn as_slice(&self) -> &[f32] {
unsafe {
std::slice::from_raw_parts(self.as_ptr(), self.len)
}
}
```

and its mutable counterpart:

```
pub fn as_mut_slice(&mut self) -> &mut [f32] {
unsafe {
std::slice::from_raw_parts_mut(self.as_mut_ptr(), self.len)
}
}
```

These operations require the implementation to guarantee that the pointer refers to a valid contiguous range containing at least `len` initialized `f32` values. The safety of these operations therefore depends on invariants established by `AlignedBuffer`.

The second unsafe boundary is the AVX implementation. Raw-pointer arithmetic and architecture-specific SIMD intrinsics require explicit unsafe operations:

```
let v = _mm256_load_ps(ptr.add(i));
```

The AVX function therefore explicitly documents the safety preconditions required by the caller, including pointer alignment, validity of the accessed memory range, and the appropriate length.

The third unsafe boundary is the C-compatible interface. For example, the destruction function is defined as:

```
pub unsafe extern "C" fn aligned_buffer_destroy(
buffer: *mut AlignedBuffer
)
```

The Rust-owned object is reconstructed using:

```
drop(Box::from_raw(buffer));
```

This operation is valid only when the pointer originated from `Box::into_raw` and has not already been destroyed.

The corresponding FFI construction function establishes this ownership relationship:

```
let buffer = Box::new(AlignedBuffer::new(size, value));
Box::into_raw(buffer)
```

The resulting FFI lifecycle can therefore be represented as:

```
aligned_buffer_create()
|
Box<AlignedBuffer>
|
Box::into_raw()
|
C/C++ owns opaque pointer
|
aligned_buffer_destroy()
|
Box::from_raw()
|
Drop
```

This provides an appropriate boundary for isolating unsafe operations, as C++ cannot express the ownership guarantees provided by Rust's type system. The FFI layer consequently acts as the bridge between the externally managed C/C++ representation and the Rust-owned object.

One issue that should nevertheless be acknowledged is the use of `assert!` in some FFI functions. For example, `aligned_buffer_len` asserts that its pointer argument is non-null. While such assertions are useful for detecting violations during development, they are not an ideal error-handling mechanism for a production C ABI. In production-oriented automotive integration, these assertions would preferably be replaced with explicit error or status handling so that failure does not depend on Rust panic behaviour at an FFI boundary.

This distinction illustrates the difference between the research prototype and a production-ready interoperability layer. The prototype uses assertions to detect violated assumptions, while a production interface would require a defined error-handling contract suitable for the target C/C++ environment.

6.3.8 Boost.Align Test Suite

Validation of the Boost.Align reimplementation was performed at both the Rust implementation level and through a C++ benchmark comparing the Boost and Rust implementations.

The Rust test suite contains tests covering the fundamental properties required by the selected implementation.

The first test verifies the alignment guarantee:

```
#[test]
fn test_alignment() {
let buffer = AlignedBuffer::new(
16,
AlignedF32x32(1.0)
);

let buffer_pointer = buffer.as_ptr() as usize;

assert_eq!(buffer_pointer % 32, 0);

}
```

The test explicitly verifies that the address returned by the buffer is divisible by 32, thereby validating the fundamental alignment requirement of the selected Boost.Align use case.

The second test validates the AVX processing path:

```
#[test]
fn test_simd() {
let mut buffer =
AlignedBuffer::new(16, AlignedF32x32(1.0));

unsafe {
    let buffer_pointer =
        buffer.as_mut_ptr() as *mut f32;

    aligned_buffer::simd::double_f32_avx(
        buffer_pointer,
        buffer.len()
    );
}

for v in buffer.as_slice() {
    assert_eq!(*v, 2.0);
}
```

```
19
20 }
```

This test simultaneously exercises aligned allocation, raw-pointer conversion, and SIMD processing. The initial value is `1.0`, and the AVX operation doubles every element, resulting in an expected value of `2.0` for each element.

In addition to the Rust tests, the project contains a C++ benchmark driver comparing the Boost.Align and Rust implementations using the same general workload. The benchmark allocates a buffer containing `2^{24} float` values, corresponding to approximately 64 MiB of data, and performs 50 processing iterations using 32-byte aligned storage.

The Boost implementation uses:

```
boost::alignment::aligned_alloc(ALIGN,
N * sizeof(float));
```

whereas the Rust implementation uses the `AlignedBuffer` abstraction.

Both implementations verify the 32-byte alignment of their allocated buffers and process the data using AVX operations. The benchmark additionally measures execution time, throughput, resident memory usage, and memory behaviour during deallocation.

The recorded benchmark results show successful correctness and alignment checks for both implementations. The reported average iteration time was approximately 21.68 ms for Boost.Align and 20.70 ms for the Rust implementation, corresponding to reported throughputs of approximately 6.19 GB/s and 6.48 GB/s, respectively. The resulting Boost-to-Rust execution-time ratio was approximately 1.047, with the Rust implementation achieving the higher measured throughput in this experiment.

The memory measurements showed different behaviour. The reported RSS increase during allocation was approximately 196 KB for the Boost implementation and 65,540 KB for the Rust implementation, while both implementations subsequently showed approximately 65,540 KB of RSS reduction during deallocation.

These results should not be interpreted as a general performance advantage of Rust over Boost.Align. The benchmark evaluates a specific 64 MiB workload, allocator configuration, AVX processing path, and experimental environment. Furthermore, the two SIMD kernels are not implemented identically at the instruction level: the C++ benchmark uses `_mm256_mul_ps` with a vector containing `2.0f`, whereas the Rust implementation uses `_mm256_add_ps(v, v)`. Although both operations produce the same mathematical result for the tested input, they represent different instructions and may have different performance characteristics.

Consequently, the benchmark primarily demonstrates that the Rust implementation can provide the required alignment and SIMD functionality with comparable performance under the tested workload. The results do not establish a general performance superiority of either implementation.

Overall, the tests demonstrate the principal objectives of the Boost.Align reimplementation. Aligned storage can be expressed through Rust's type system, the resulting buffer can participate in an ownership-based memory model, and the aligned memory can be exposed to low-level SIMD operations and C++ through a narrowly defined unsafe/FFI boundary. This provides the required functionality for the selected use case while avoiding a direct reproduction of the original Boost allocation API.

6.4 Boost.Endian Reimplementation

The Boost.Endian reimplementation follows the three complementary approaches provided by the original library: endian conversion functions, endian buffer types, and endian arithmetic types. This structure was selected because the three approaches represent different trade-offs between explicit control over byte-order conversion, convenience, and arithmetic support. Boost.Endian defines these approaches as separate abstractions rather than simply providing a single endian-aware integer type.

The Rust implementation preserves this distinction. The `endian` crate is divided into three principal modules:

```
1 conversion.rs
2 buffer.rs
3 arithmetic.rs
```

A fourth module, `ffi.rs`, provides interoperability with C++ through the `cxx` crate.

The implementation follows the same conceptual organization as Boost.End, adapting the interfaces to Rust's type system and standard library facilities.

The three Rust approaches are:

1. **Conversion functions**, where ordinary Rust integer and floating-point values are explicitly converted between byte orders.
2. **Buffer types**, where the desired byte order is associated with a type and conversion to and from the native representation is explicit.
3. **Arithmetic types**, where the value is stored in a specified byte order while arithmetic and bitwise operations are provided directly on the endian-aware type.

The organization is the same as the design described in the Boost.Endian documentation. Conversion functions provide explicit conversion operations, buffer types provide explicitly

converted storage and arithmetic types provide implicit conversion together with arithmetic operations.

The Rust implementation additionally exposes the functionality to C++ through a cxx bridge. This allows the existing C++ code to use the Rust implementation without requiring the C++ application to depend directly on Rust-specific representations.

6.4.1 Scope and API Mapping

Boost.Endian provides a considerably broader API. The original library supports integer sizes from 8 to 64 bits, including non-standard sizes such as 24, 40 and 48 bits, together with aligned and unaligned buffer and arithmetic types. It also provides conversion functions and support for user-defined types.

The Rust implementation reproduces the three principal approaches but does not attempt to reproduce every Boost.Endian typedef and ABI detail.

The implementation mapping is the following:

Boost.Endian concept	Rust implementation	Purpose
`endian_reverse`	`endian_reverse`	Reverse the byte representation
`big_to_native`	`big_to_native`	Convert big-endian representation to native order
`native_to_big`	`native_to_big`	Convert native representation to big-endian order
`little_to_native`	`little_to_native`	Convert little-endian representation to native order
`native_to_little`	`native_to_little`	Convert native representation to little-endian order
In-place conversions	`_inplace` functions	Modify an existing value
Endian load/store	`load_` / `store_`	Convert between byte arrays and native values
`endian_buffer`	`EndianBuf<E,T>`	Explicit endian-aware storage
Buffer typedefs	`BigEndianBuf<T>`, `LittleEndianBuf<T>`	Convenient typed interfaces
`endian_arithmetic`	`EndianInt<E,T>`	Endian-aware arithmetic type
Arithmetic typedefs	`BigEndianInt<T>`, `LittleEndianInt<T>`	Convenient arithmetic types
C++ interoperability	`ffi` / `cxx`	C++ access to Rust functionality

TABLE 6.3: Mapping between Boost.Endian concepts and their Rust implementation.

The implementation therefore preserves the architectural model of Boost.Endian without requiring a literal translation of the C++ template hierarchy.

The root crate also defines Boost.Endian-style aliases such as:

```
pub type BigInt32BufT = BigEndianBuf<i32>;
pub type BigUInt32BufT = BigEndianBuf<u32>;

pub type BigInt32T = BigEndianInt<i32>;
pub type BigUInt32T = BigEndianInt<u32>;
```

This provides an interface that is recognizable to users familiar with the Boost library while retaining Rust's generic and trait-based implementation model.

The original Boost library explicitly distinguishes buffer types from arithmetic types because buffer conversions are explicit whereas arithmetic conversions are implicit. This distinction is preserved in the Rust design.

6.4.2 Endian Representation

The fundamental design decision in the Rust implementation is to represent byte ordering through textbfcompile-time encoding types.

Three marker types are defined:

```
pub struct BigEndian;
pub struct LittleEndian;
pub struct NativeEndian;
```

These types implement the `Encoding` trait:

```
pub trait Encoding: Copy {
fn to_stored<T: EndianReversible>(native: T) -> T;
fn to_native<T: EndianReversible>(stored: T) -> T;
}
```

The marker type therefore determines how a value is converted to and from its stored representation.

For example, the big-endian implementation performs a conversion only when the host machine is little-endian:

```
impl Encoding for BigEndian {
fn to_stored<T: EndianReversible>(v: T) -> T {
if NATIVE_ORDER == Order::Big {
v
} else {
v.endian_reverse()
}
}
}
```

The native byte order is determined at compilation using Rust's target configuration:

```
pub const NATIVE_ORDER: Order = {
#[cfg(target_endian = "big")]
{
Order::Big
```

```
}
#[cfg(target_endian = "little")]
{
Order::Little
}
};
```

This means that the decision whether a byte swap is required is resolved according to the target architecture rather than being detected dynamically at runtime.

The actual byte reversal is abstracted through the `EndianReversible` trait:

```
pub trait EndianReversible: Sized + Copy {
fn endian_reverse(self) -> Self;
}
```

The trait is implemented for Rust's primitive integer types and for `f32` and `f64`.

For integer types, Rust's native `swap_bytes()` operation is used:

```
impl EndianReversible for u32 {
fn endian_reverse(self) -> Self {
self.swap_bytes()
}
}
```

Floating-point values are handled by converting the floating-point value to its bit representation, reversing those bits as a byte sequence, and converting the result back:

```
Self::from_bits(self.to_bits().swap_bytes())
```

The operation is therefore a transformation of the binary representation rather than a numerical floating-point operation.

This design is consistent with the purpose of Boost.Endian, where the objective is to manipulate the representation of arithmetic values independently of the host architecture. Boost.Endian also supports floating-point types in its more recent buffer and arithmetic interfaces.

6.4.3 Endian Conversion Functions

The first implementation approach corresponds to Boost.Endian's conversion-function interface.

The Rust implementation provides:

```
endian_reverse()
big_to_native()
native_to_big()
little_to_native()
native_to_little()
```

together with their in-place variants:

```
endian_reverse_inplace()
big_to_native_inplace()
native_to_big_inplace()
little_to_native_inplace()
native_to_little_inplace()
```

These functions operate directly on ordinary Rust values.

For example:

```
let value = 0x01020304u32;

let reversed = endian_reverse(value);
```

produces:

```
0x04030201
```

The conversion between a specified byte order and the native byte order is implemented conditionally:

```
pub fn big_to_native<T: EndianReversible>(x: T) -> T {
if NATIVE_ORDER == Order::Big {
x
} else {
x.endian_reverse()
}
}
```

The reverse conversion is symmetric:

```
pub fn native_to_big<T: EndianReversible>(x: T) -> T {
big_to_native(x)
}
```

The same structure is used for little-endian conversion.

This provides an important property: when the requested representation already matches the native byte order, the conversion does not perform a byte swap.

The implementation also provides a conditional conversion function:

```
conditional_reverse(x, from, to)
```

which resolves `Order::Native` and reverses the value only when the source and destination representations differ.

This corresponds to the Boost.Endian design in which conversion functions are intended for applications where the programmer wants explicit control over when conversion occurs. Boost documents both mutating and non-mutating conversion operations, including conditional variants.

Load and Store Operations

In addition to conversions between native values, the implementation provides explicit load and store operations.

For example:

```
pub fn load_big_u32(p: &[u8; 4]) -> u32 {
u32::from_be_bytes(*p)
}
```

and:

```
pub fn store_big_u32(p: &mut [u8; 4], v: u32) {
*p = v.to_be_bytes();
}
```

Equivalent functions are provided for little-endian values and for signed and unsigned 16-, 32- and 64-bit integers.

These functions provide a direct abstraction for binary data processing:

```
byte representation
|
v
load_big_u32()
|
v
native u32
|
v
application processing
|
v
store_big_u32()
|
v
byte representation
```

The use of fixed-size arrays such as `[u8; 4]` is significant because the size requirement is represented directly in the Rust type system. A four-byte load cannot accidentally receive a three-byte array.

24-bit Values

The implementation additionally provides 24-bit load and store functions:

```
load_big_u24()
load_big_s24()
load_little_u24()
load_little_s24()

store_big_u24()
store_big_s24()
store_little_u24()
store_little_s24()
```

Unsigned 24-bit values are represented using `u32`, while signed 24-bit values are represented using `i32`.

For signed values, the implementation explicitly performs sign extension:

```
if raw & 0x0080_0000 != 0 {
(raw | 0xFF00_0000) as i32
} else {
raw as i32
```

```
}
```

This is required because Rust, like C++, does not provide a native 24-bit integer type.

The implementation therefore preserves the external three-byte representation while using a standard 32-bit integer internally.

This is particularly relevant because one of the motivations for Boost.Endian is support for binary formats whose integer field sizes do not correspond directly to native C++ integer sizes. The original library supports 8-, 16-, 24-, 32-, 40-, 48-, 56- and 64-bit buffer and arithmetic representations.

The current C++ `cxx` bridge, however, exposes the 16-, 32- and 64-bit load/store operations rather than the 24-bit Rust functions. Thus, the 24-bit functionality exists in the Rust implementation but is not currently part of the demonstrated C++ interoperability surface.

6.4.4 Endian Buffer Types

The second approach is implemented through the generic type:

```
pub struct EndianBuf<E: Encoding, T: EndianReversible> {
stored: T,
_enc: PhantomData<E>,
}
```

The encoding is represented by the type parameter `E`, while `T` specifies the underlying value type.

Three convenient aliases are provided:

```
pub type BigEndianBuf<T> =
EndianBuf<BigEndian, T>;

pub type LittleEndianBuf<T> =
EndianBuf<LittleEndian, T>;

pub type NativeEndianBuf<T> =
EndianBuf<NativeEndian, T>;
```

A value is constructed from a native-endian value:

```
let buffer: BigEndianBuf<u32> =
BigEndianBuf::new(0x01020304);
```

The constructor immediately converts the value into the requested stored representation:

```
Self {
stored: E::to_stored(native),
_enc: PhantomData,
}
```

The conversion back to native order occurs explicitly through `get()`:

```
pub fn get(self) -> T {
E::to_native(self.stored)
}
```

Values can be replaced using:

```
pub fn set(&mut self, native: T)
```

This preserves the fundamental semantic difference between the Boost.Endian buffer and arithmetic approaches.

With the buffer representation, conversion is explicit:

```
let mut value: BigEndianBuf<u32> =
BigEndianBuf::new(10);

value.set(20);

let native = value.get();
```

The type itself does not automatically participate in arithmetic operations.

The implementation uses:

```
#[repr(transparent)]
```

for `EndianBuf`.

This makes the wrapper have the same representation characteristics as its stored value for ABI purposes. The project also contains explicit tests verifying that:

```
size_of::<BigEndianBuf<u32>>()
== size_of::<u32>()
```

and:

```
size_of::<LittleEndianBuf<i64>>()
== size_of::<i64>()
```

The stored value is already converted to the target byte order, meaning that its memory representation can be used directly for binary I/O.

This corresponds closely to the purpose of Boost.Endian buffer types. Boost describes an endian buffer as a byte-holder whose byte order is independent of the native architecture and whose conversions are explicitly controlled by the programmer.

Equality and Ordering

The Rust implementation deliberately distinguishes equality from ordering.

Equality compares the stored representation:

```
self.stored == other.stored
```

while ordering converts the values to native representation:

```
self.get().cmp(&other.get())
```

This allows ordering to represent the numerical meaning of the stored values rather than their byte representation.

Hashing is performed on the stored representation:

```
self.stored.hash(state)
```

which maintains consistency with the equality implementation.

The buffer type also implements standard Rust traits including `Clone`, `Copy`, `Default`, `Debug` and `Display` where the underlying type supports the corresponding trait.

6.4.5 Endian Arithmetic Types

The third approach is implemented by:

```
pub struct EndianInt<E: Encoding, T: EndianReversible> {
stored: T,
_enc: PhantomData<E>,
```

```
}
```

This structure is conceptually similar to `EndianBuf`, but additionally implements arithmetic and bitwise operators.

The two primary aliases are:

```
pub type BigEndianInt<T> =
EndianInt<BigEndian, T>;

pub type LittleEndianInt<T> =
EndianInt<LittleEndian, T>;
```

Native-endian arithmetic types are also provided:

```
pub type NativeEndianInt<T> =
EndianInt<NativeEndian, T>;
```

The important difference from the buffer type is the conversion model.

Construction can occur implicitly:

```
let mut x: BigEndianInt<i32> = 100.into();
```

and conversion back to the native value is also supported through `From`:

```
let value: i32 = x.into();
```

The implementation additionally provides:

```
x.get()
```

for explicit retrieval.

Arithmetic Operators

The implementation provides the standard binary arithmetic operations:

```
+
-

```

```
*

/
%
```

as well as bitwise operations:

```
&
|
^
<<

> >
```

Both endian-aware values and native scalar values are supported.

For example:

```
let mut x: BigEndianInt<i32> = 100.into();

x += 42;
x *= 2;
x -= 84;
x /= 4;
```

Each operation retrieves the native representation, performs the operation using the underlying Rust type and stores the result back into the selected byte order.

The implementation follows this pattern:

```
Self::new(
self.get().$method(rhs.get())
)
```

for operations involving two endian-aware values.

Assignment operations follow the equivalent approach:

```
self.set(
self.get().$base_method(rhs.get())
);
```

Unary `Neg` and `Not` operations are also implemented.

Rust does not provide C++'s `++` and `--` operators, so the implementation instead exposes:

```
x.inc();
x.dec();
```

These provide the equivalent semantic operation.

This is an example of where the implementation preserves the functionality rather than reproducing the C++ syntax exactly.

Boost.Endian's arithmetic types are specifically intended to provide the same general operations as ordinary arithmetic types while performing the necessary endian conversion implicitly.

Comparison and Ordering

The arithmetic type implements `PartialEq`, `Eq`, `PartialOrd` and `Ord`.

Unlike raw byte comparison, ordering is performed on the native numerical values:

```
self.get().cmp(&other.get())
```

This ensures that:

```
let a: BigEndianInt<i32> = 10.into();
let b: BigEndianInt<i32> = 20.into();

assert!(a < b);
```

has the expected numerical semantics.

The implementation also provides `Hash`, `Clone`, `Copy`, `Default`, `Debug` and `Display` where appropriate.

Conversion Hoisting

The project explicitly demonstrates the performance consideration documented by Boost.Endian: repeated implicit conversions inside computational loops can be avoided by converting the value once, performing the computation using the native value, and storing the result back afterwards.

The C++ demonstration contains the equivalent pattern:

```
int32_t native_v = int32_t(big_v);

for (int32_t i = 0; i < 1000; ++i)
native_v += i;

big_v = native_v;
```

The demonstration therefore reproduces one of the optimization patterns discussed in the Boost.Endian documentation. Boost notes that arithmetic types can perform repeated implicit conversions and that conversions can be hoisted out of performance-critical loops when appropriate.

6.4.6 C++ Interoperability

C++ interoperability is implemented through the `cxx` crate rather than a manually constructed `extern "C"` interface.

The crate declares a C++11-compatible bridge using:

```
#[cxx::bridge(namespace = "boost_endian")]
pub mod ffi
```

The build script invokes:

```
cxx_build::bridge("src/ffi.rs")
.std("c++11")
.compile("endian_bridge");
```

The resulting bridge generates C++ headers and a bridge library.

The Rust crate itself is built as a static library:

```
[lib]
name = "endian"
path = "src/lib.rs"
crate-type = ["staticlib"]
```

The C++ application can consequently link against the Rust library and generated bridge.

The project documents the corresponding build sequence:

```
cargo build --release
|
v
generated cxx bridge
|
v
libendian.a
libendian_bridge.a
|
v
g++ C++ application
```

The C++ application includes:

```
#include "rust/cxx.h"
#include "boost-endian/src/ffi.rs.h"
```

and uses the functions exposed by the bridge in the `boost_endian` namespace.

Conversion Interface

The FFI exposes explicit functions for:

- byte reversal;
- big-endian/native conversion;
- native/big-endian conversion;
- little-endian/native conversion;
- native/little-endian conversion;
- loading values from byte slices; and
- storing values into byte slices.

For example:

```
uint32_t value =
load_big_u32(
rust::Slice<const uint8_t>(buf, 4)
);
```

and:

```
store_big_u32(
rust::Slice<uint8_t>(buf, 4),
0xCAFEBABE
);
```

The use of `rust::Slice` allows the C++ caller to provide an existing memory region without transferring ownership of the memory to Rust.

Buffer Interface

The bridge also exposes shared structures such as:

```
struct BigInt32Buf {
raw: i32
}
```

and:

```
struct LittleUInt64Buf {
raw: u64
}
```

The `raw` member contains the value in the selected target byte order.

C++ can therefore create and retrieve values through:

```
BigInt32Buf b =
big_int32_buf_new(0x01020304);

int32_t value =
big_int32_buf_get(b);
```

The demonstration additionally inspects the raw representation to verify that a big-endian value has its most significant byte first and a little-endian value has its least significant byte first on the tested little-endian host.

Arithmetic Interface

The `cxx` bridge itself does not expose Rust operator overloading directly to C++. Consequently, the project implements a thin C++ template on top of the bridge functions.

The template stores an FFI buffer and performs operations by:

1. retrieving the native value;
2. performing the C++ operation; and
3. converting the result back to the endian buffer.

This provides a C++ interface resembling Boost.Endian's arithmetic types without attempting to expose Rust's operator traits directly through the FFI.

This is an important example of semantic adaptation rather than literal translation: Rust's `Add`, `Sub`, `Mul` and assignment traits are appropriate within Rust, while a C++ template is used to reproduce the corresponding C++ programming model on the other side of the interoperability boundary.

6.4.7 Unsafe Code isolation

A major difference between the Boost.Endian and Boost.Align implementations is that the endian implementation requires substantially less `unsafe` Rust code.

The core endian operations are implemented using safe Rust primitives:

```
swap_bytes()
to_be_bytes()
from_be_bytes()
to_le_bytes()
from_le_bytes()
copy_from_slice()
```

No raw-pointer manipulation is required for the Rust-native conversion, buffer, or arithmetic abstractions.

The load functions use fixed-size Rust arrays and indexing:

```
u32::from_be_bytes([
p[0], p[1], p[2], p[3]
])
```

while the store functions use:

```
p[..4].copy_from_slice(
&v.to_be_bytes()
);
```

The safety properties of these operations are therefore enforced by Rust's slice and array abstractions.

The FFI boundary uses `cxx bridge` and `rust Slice` rather than manually reconstructing Rust slices from raw C++ pointers. This significantly reduces the amount of explicit `unsafe` code needed for interoperability.

The C++ caller is nevertheless responsible for providing valid buffers of the required size. The Rust functions index the supplied slices directly, so an undersized slice results in a Rust bounds failure. The FFI module explicitly documents this precondition:

```
p must be a slice of at least N bytes
```

This is an important boundary condition for production integration. The current implementation relies on Rust's bounds checking rather than exposing unchecked raw-pointer access, but the error path is not converted into an explicit C++ error or status value.

The implementation comments also acknowledge that a panic resulting from an undersized slice crosses the FFI boundary through the `cxx` machinery and is not intended to be used as a normal error-reporting mechanism.

For the research implementation, this provides a relatively small safety boundary. For production automotive integration, invalid input lengths would preferably be rejected before invoking the Rust function or represented through an explicit fallible interface.

Overall, the endian implementation demonstrates that much of the functionality traditionally implemented through low-level byte manipulation in C++ can be expressed using safe Rust primitives.

This directly supports one of the migration objectives established in 4: preserve the relevant library behaviour while minimizing and isolating `unsafe` Rust, particularly around interoperability boundaries.

6.4.8 Boost.Endian Test Suite

The Rust implementation contains unit tests for each of the three approaches.

Conversion Tests

The conversion module tests byte reversal:

```
assert_eq!(
endian_reverse(0x01020304u32),
0x04030201u32
);
```

Round-trip tests verify that converting to an endian representation and back produces the original value:

```
let v: u32 = 0xDEAD_BEEF;

assert_eq!(
big_to_native(native_to_big(v)),
v
);
```

The same pattern is tested for little-endian 64-bit values.

The load/store interface is also tested:

```
let mut buf = [0u8; 4];

store_big_u32(
&mut buf,
0xCAFE_BABE
);

assert_eq!(
load_big_u32(&buf),
0xCAFE_BABE
);
```

A dedicated test validates signed 24-bit sign extension:

```
store_big_s24(&mut buf, -1);
assert_eq!(load_big_s24(&buf), -1);

store_little_s24(&mut buf, -42);
assert_eq!(load_little_s24(&buf), -42);
```

Floating-point byte reversal is also validated through a double-reversal test:

```
let v = std::f32::consts::PI;

assert_eq!(
endian_reverse(endian_reverse(v)),
v
);
```

Finally, the in-place implementation is compared against the corresponding value-returning implementation.

Buffer Tests

The buffer implementation tests:

- big-endian round trips;
- little-endian round trips;
- byte representation;
- equality;
- ordering;
- mutation through `set()`; and
- size equivalence with the underlying primitive type.

For example:

```
let buf: BigEndianBuf<u32> =
BigEndianBuf::new(0xDEAD_BEEF);

assert_eq!(
buf.get(),
0xDEAD_BEEF
);
```

The size test is particularly relevant to interoperability:

```
assert_eq!(
size_of::<BigEndianBuf<u32>>(),
size_of::<u32>()
);
```

This verifies the expected transparent representation.

Arithmetic Tests

The arithmetic implementation validates implicit construction, arithmetic assignment, ordering, size, and repeated accumulation.

For example:

```
let mut x: BigEndianInt<i32> =
100.into();

x += 42;

assert_eq!(
x.get(),
142
);
```

The test suite also performs a Gauss-sum style workload:

```
let mut x: BEi32 = 0.into();

for i in 1..=100_i32 {
x += i;
}

assert_eq!(x.get(), 5050);
```

This tests the interaction between repeated arithmetic operations and endian conversion.

C++ Integration Tests

The C++ demonstration exercises all three approaches.

The conversion tests verify:

```
endian_reverse
native -> big -> native
native -> little -> native
store -> load
```

The buffer tests verify both the numerical value and the underlying byte representation.

A mixed-endian structure is also constructed:

```
struct Header {
BigInt32Buf     file_code;
BigInt32Buf     file_length;
LittleInt32Buf  version;
LittleInt32Buf  shape_type;
};
```

The test confirms that the big- and little-endian fields have different byte representations while retaining their correct numerical values.

A second example models a mixed-endian binary record:

```
struct Record {
BigUInt32Buf    magic;
LittleUInt16Buf seq;
BigInt32Buf     payload_len;
};
```

The resulting structure is checked with a `static_assert`, and its binary representation is printed and inspected.

This demonstrates a practical use case for endian-aware data structures: a single record may contain fields with different externally specified byte orders.

FFI Overhead Benchmark

The repository additionally contains a dedicated FFI benchmark.

The benchmark compares two implementations:

```
C++ Native
manual big-endian byte shifts

Rust FFI
cxx bridge load/store
```

The benchmark operates on arrays of 64-bit values and executes 50 iterations for each tested input size.

The tested element counts are:

```
2^8
2^14
2^20
2^24
```

corresponding to approximately:

```
2 KiB
128 KiB
8 MiB
128 MiB
```

of data per iteration.

For the native C++ implementation, the benchmark manually reconstructs a 64-bit value from eight bytes and subsequently writes the incremented value back to the byte array.

The Rust implementation performs the equivalent operation through:

```
load_big_u64(...)
```

followed by:

```
store_big_u64(...)
```

through the `cxx` bridge.

The benchmark records:

- total execution time;
- time per iteration in milliseconds;
- time per iteration in nanoseconds;
- throughput in GB/s; and
- correctness.

The benchmark also computes the relative overhead:

$$\text{overhead} = \left(\frac{T_{\text{Rust FFI}}}{T_{\text{C++}}} - 1 \right) \times 100$$

This allows the cost of crossing the Rust/C++ interface to be evaluated independently of functional correctness.

The repository's `endian_comparison.md` also documents several implementation variants developed during the project. It identifies the original `endian` crate as the Rust-native baseline and describes later variants based on different C/C++ interoperability approaches, including a `cxx`-based implementation. This provides useful experimental context, while the implementation described in this section corresponds to the `endian` crate contained in the repository together with its current `cxx` integration.

6.5 Evaluation and Critical Analysis

The evaluation is designed to answer the research questions without extending the evidence beyond what the experiments can establish. Functional tests assess whether the selected behaviour is reproduced. The benchmarks assess observed execution time, throughput, memory observations, and the cost of selected FFI paths. The unsafe-code analysis identifies where Rust's static guarantees stop applying and where correctness depends on explicit interface contracts.

6.5.1 Evaluation Methodology

The reference implementation is the corresponding Boost implementation where a direct comparison is available. Each benchmark uses release builds and 50 iterations per tested configuration. The experiments are intended to characterize the selected workloads rather than establish universal performance properties. Results are therefore reported as measurements for the experimental environment.

6.5.2 Boost.Align Performance Evaluation

The end-to-end Boost.Align benchmark uses a 64 MB buffer containing 16,777,216 `f32` values and 50 iterations. It measures allocation, initialization, the SIMD operation, and deallocation. Both implementations passed the functional correctness and 32-byte alignment checks.

Metric	**Boost.Align**	**Rust AlignedBuffer**
Total time	1.0839 s	1.0351 s
Time per iteration	21.6782 ms	20.7014 ms
Throughput	6.19 GB/s	6.48 GB/s
Correctness	Pass	Pass
32-byte alignment	Pass	Pass

TABLE 6.4: Boost.Align end-to-end benchmark results

The measured execution time is approximately 4.7% lower for the Rust implementation in this workload. This is an observed result, not evidence that Rust is generally faster than Boost.Align.

The memory table reports changes in process-level memory observations rather than the total size of the allocated buffer. The 64 MiB benchmark buffer corresponds to approximately 65,536 KB, which is consistent with the approximately 65,540 KB increase in virtual memory reported for both implementations. Virtual memory describes the process address space made available to the process, whereas RSS describes the portion observed as resident in physical memory at the measurement point. Consequently, the +196 KB RSS value for

Boost.Align should not be interpreted as meaning that only 196 KB was allocated for the benchmark buffer; it is the observed RSS change at that measurement point. The substantially larger Rust RSS change indicates different observed residency behaviour during allocation. The measurements are therefore useful for describing the behaviour of these two implementations in this environment, but they do not establish a general memory-efficiency advantage for either implementation.

Memory metric	**Boost.Align**	**Rust AlignedBuffer**
RSS delta on allocation	+196 KB	+65,540 KB
Virtual-memory delta	+65,540 KB	+65,540 KB
RSS delta during run	+64 KB	0 KB
RSS delta on free	-65,540 KB	-65,540 KB

TABLE 6.5: Observed memory behaviour in the Boost.Align benchmark

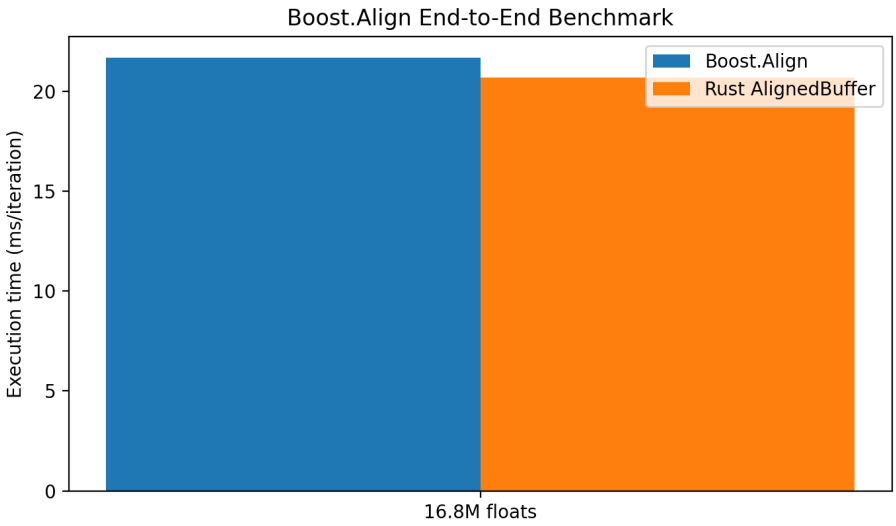


FIGURE 6.1: Measured Boost.Align end-to-end execution time

6.5.3 Boost.Align FFI Overhead

A separate benchmark isolates the effect of the Rust/C++ call path by comparing native C++, Rust through the `AlignedBuffer` abstraction, and Rust through a raw-pointer interface. For the smallest tested buffer, the measured times were 148 ns, 146 ns, and 137 ns respectively. For the largest tested buffer, the corresponding measurements were 7,110,144 ns, 7,103,436 ns, and 6,981,330 ns.

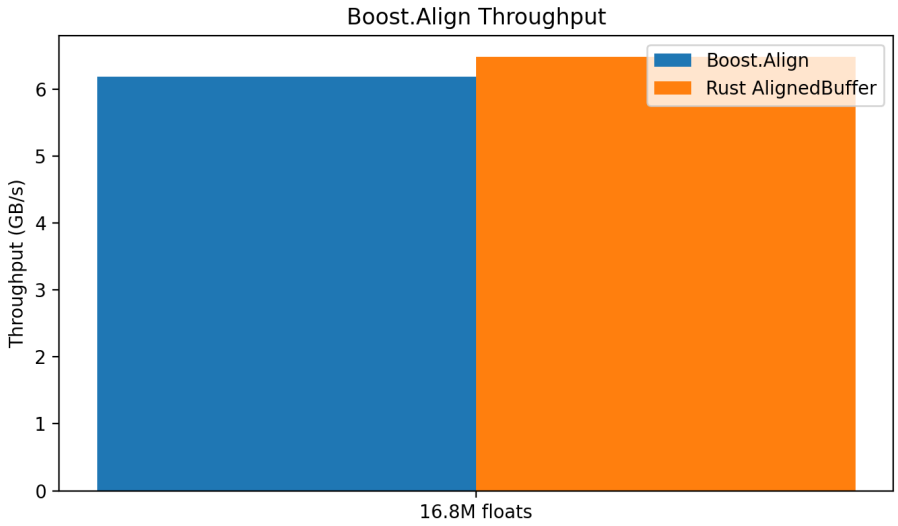


FIGURE 6.2: Measured Boost.Align throughput

Buffer	C++ native	Rust FFI (AlignedBuffer)	Rust FFI (raw pointer)
256 floats	148 ns	146 ns	137 ns
16,777,216 floats	7,110,144 ns	7,103,436 ns	6,981,330 ns

TABLE 6.6: Selected Boost.Align FFI-overhead benchmark results

The relative differences are small compared with the total operation for the large buffer. At the smallest size, the fixed cost of the interface and surrounding benchmark operations represents a larger fraction of the measurement. The experiment therefore supports the narrower conclusion that the tested FFI path did not introduce a large measured overhead for the selected operation; it does not establish that FFI overhead is negligible for arbitrary call frequencies or workloads.

6.5.4 Boost.Endian Performance Evaluation

The Boost.Endian benchmark compares a native C++ implementation using manual big-endian byte shifts with the Rust implementation called through the `cxx` bridge. The experiment uses four input sizes and 50 iterations per size.

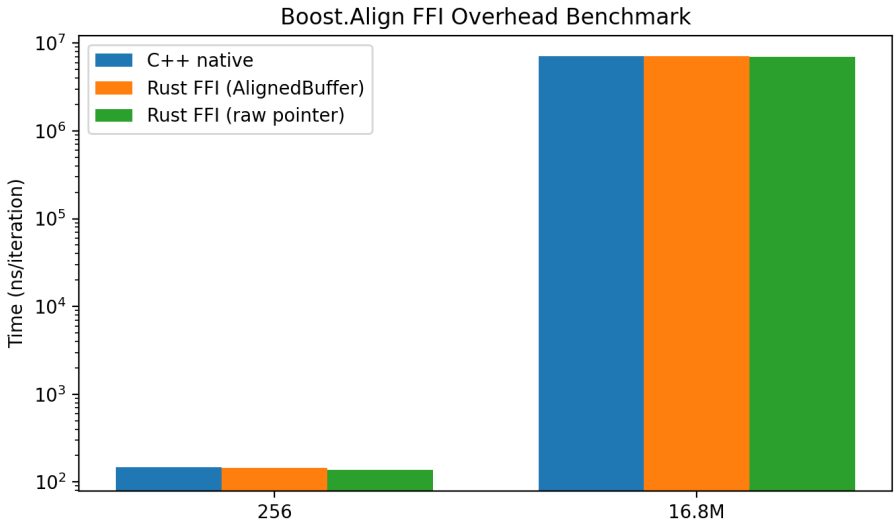


FIGURE 6.3: Selected Boost.Align FFI-overhead measurements; logarithmic vertical scale

Elements	C++ ns/iter	Rust ns/iter	C++ GB/s	Rust GB/s
256	1,406	880	2.91	4.66
16,384	97,331	62,995	2.69	4.16
1,048,576	5,884,663	3,842,808	2.85	4.37
16,777,216	92,625,204	60,191,448	2.90	4.46

TABLE 6.7: Boost.Endian FFI benchmark results

The Rust implementation has lower measured execution time in all four tested cases, with the reported average speedup of approximately 35%. However, the comparison baseline is a manual C++ byte-manipulation implementation rather than the original `Boost.Endian` implementation. The result therefore demonstrates performance characteristics of the selected Rust implementation against that particular baseline, not a general performance advantage over Boost.Endian.

6.5.5 Unsafe Code Inventory

The migration deliberately distinguishes safe Rust code from operations for which Rust requires explicit `unsafe`. In Boost.Align, the unsafe surface is concentrated around raw-slice construction, AVX operations using raw pointers and architecture-specific intrinsics, and the C-compatible destruction interface. In Boost.Endian, the core conversion and representation operations use safe Rust primitives, while the C++ interoperability layer is mediated by `cxx`.

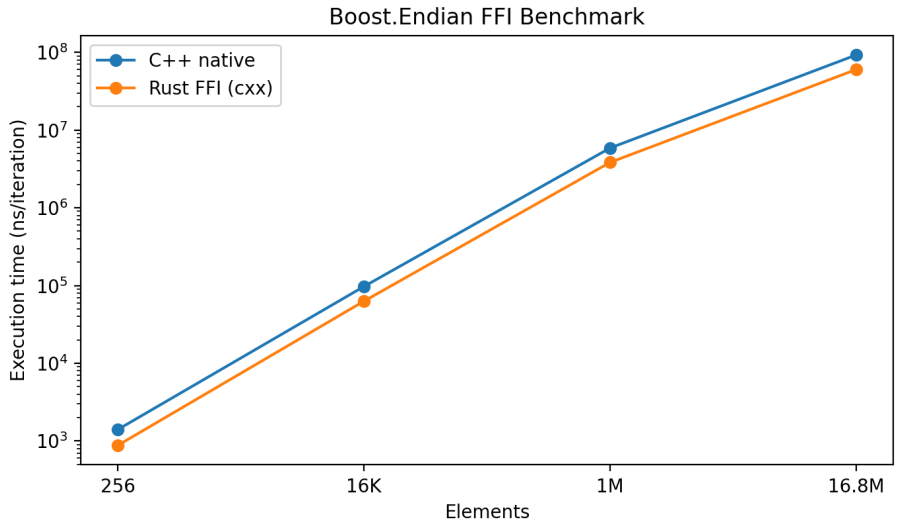


FIGURE 6.4: Boost.Endian measured execution time across input sizes

The safety analysis distinguishes the number and location of unsafe regions from the measured source-level line counts. The repository used for the implementation and benchmark results was inspected directly, excluding generated `target/` artifacts. Across the tracked Rust files, there are 1,066 non-blank, non-comment source lines. The implementation source files (`src/` and build scripts) account for 990 lines, while the Rust test files account for 76 lines.

For this analysis, unsafe LOC is defined as non-blank, non-comment lines occurring inside an `unsafe fn` body or an explicit `unsafe` block. By this definition, the implementation contains 51 unsafe LOC: 51 in Boost.Align and 0 in Boost.Endian. The test suite contains a further 4 unsafe LOC used to invoke the AVX implementation directly. Boost.Align contains 9 unsafe functions and 2 explicit unsafe blocks in the implementation; Boost.Endian contains none. These measurements show that unsafe code is concentrated in the Boost.Align pointer, SIMD, and C-compatible interface rather than being distributed throughout the data-model and conversion abstractions.

Component	Unsafe functions	Unsafe blocks	Unsafe LOC	Safety invariants
Boost.Align	9	2	51	Raw slice validity; pointer validity and alignment; SIMD pointer/length assumptions; FFI destruction ownership
Boost.Endian	0	0	0	Slice bounds and fixed-size representations are handled by safe Rust abstractions; FFI input validity remains a contract
Rust tests	0	1	4	The test directly invokes the AVX implementation through an unsafe block; the test establishes the required buffer alignment and length conditions

TABLE 6.8: Measured unsafe-code inventory in the Rust implementation and tests

The measured unsafe surface is therefore localized rather than spread throughout the data-model and conversion abstractions. This localization does not remove the need for review: each unsafe region depends on invariants concerning pointer validity, alignment, length, aliasing, ownership, and lifetime. The LOC measurement uses the same repository revision as the supplied implementation and benchmark artifacts and excludes generated `target/` files.

6.5.6 Safety Guarantees Across the FFI Boundary

Rust's static guarantees apply most strongly inside the Rust type system. At an FFI boundary, several properties must instead be established by the interface contract and by the foreign caller. Table 6.9 summarizes this distinction.

Property	**Inside Rust**	**Across FFI**
Ownership	Type-system assisted	Contract / caller responsibility
Lifetime	Type-system assisted	Contract / caller responsibility
Aliasing	Type-system assisted	Must be respected by foreign caller
Bounds	Slice/reference checks	Input validity must be established
Alignment	Type/layout guarantees	Foreign caller must satisfy contract
Data layout	Rust representation rules	ABI contract required
Panic/exception propagation	Rust-controlled	Must be prevented or explicitly represented
Thread-safety assumptions	Rust types can constrain them	Foreign code must respect the interface contract

TABLE 6.9: Safety properties inside Rust and at the FFI boundary

This distinction is central to the proposed migration methodology. Replacing a C++ component with Rust does not automatically make the surrounding hybrid system memory-safe. The migrated component can benefit from Rust's static guarantees internally, while the boundary remains a trusted interface whose assumptions must be documented, reviewed, and tested.

6.5.7 Generalizable and Case-Specific Migration Decisions

The two implementations allow migration decisions to be separated into general patterns and case-specific choices.

Decision	Align-specific	Endian-specific	Generalizable
Semantic rather than syntactic translation			Yes
Explicit ownership/lifetime contracts			Yes
Unsafe-code isolation			Yes
Minimal FFI surface			Yes
Validation against reference behaviour			Yes
32-byte alignment	Yes		
AVX intrinsics	Yes		
C++03 compatibility constraint	Yes		
C ABI	Yes		
Fixed byte-order representations		Yes	
`cxx` bridge		Yes	

TABLE 6.10: Generalizable and case-specific migration decisions

The generalizable contribution is therefore a decision process rather than a fixed implementation recipe. A future migration should repeat the semantic analysis and interface-design steps, but it should not automatically reproduce the C ABI, `cxx`, alignment strategy, or representation choices used in these two case studies.

6.5.8 Critical Analysis of the Contribution

Several migration patterns emerge from the implementation. First, the migration is more accurately described as semantic adaptation than source translation: the Rust implementation preserves relevant observable behaviour while using ownership, traits, standard-library representations, and other Rust-native abstractions. Second, unsafe operations are most useful when concentrated into small regions with explicit invariants. Third, FFI design must be treated as an architectural decision rather than as a final binding step. Fourth, compatibility requirements can determine the FFI mechanism: the Align case requires a C++03-compatible boundary, whereas the Endian case can use `cxx`. Finally, evaluation must distinguish functional equivalence from performance, memory behaviour, and safety guarantees because each requires different evidence.

The experiments also expose trade-offs rather than a single direction of improvement. Boost.Align shows a small measured execution-time difference in favour of the Rust implementation for the selected AVX workload, but a substantially larger observed RSS increase at allocation. Boost.Endian shows lower measured execution time than the manual C++ baseline,

but that baseline is not the original Boost.Endian implementation. These results demonstrate why claims about performance or memory improvements must remain workload- and implementation-specific.

The contribution should consequently be understood as a feasibility and methodology study for selected low-level Boost functionality. It does not establish that arbitrary template-heavy Boost libraries can be migrated automatically, that Rust universally improves performance or memory consumption, or that a hybrid Rust/C++ architecture is automatically suitable for a particular ISO 26262 safety case.

6.5.9 Limitations of the Evaluation

The evaluation has several limitations. The experiments use a limited number of workloads and a single development environment. The Align benchmark compares different implementation details, and the Endian benchmark compares Rust against a manual C++ baseline rather than directly against Boost.Endian. The measurements report observed execution time and throughput but do not provide worst-case execution time, jitter, schedulability, or hard real-time evidence. The RSS observations are process-level measurements and should not be interpreted as a comprehensive characterization of allocator behaviour. Finally, the functional tests establish correctness for the selected cases but do not constitute exhaustive verification of the complete Boost APIs.

These limitations define the boundary of the claims made by this dissertation and motivate the future validation activities presented in Chapter 7.

6.5.10 Implementation Summary

The Boost.Endian reimplementation demonstrates that the three conceptual approaches provided by the original C++ library can be mapped naturally onto Rust abstractions.

The resulting architecture can be summarized as follows:

```
Boost.Endian
|
+----------------+----------------+
|                |                |
Conversion          Buffer          Arithmetic
Functions           Types             Types
|                |                |
v                v                v
conversion.rs     buffer.rs       arithmetic.rs
|                |                |
+----------------+----------------+
|
EndianReversible
```

```
|
v
Native Rust types
|
v
ffi.rs
|
cxx bridge
|
v
C++ code
```

The conversion layer provides explicit byte-order transformations and binary load/store functions. The buffer layer associates a fixed byte order with stored values while requiring explicit conversion to native values. The arithmetic layer adds operator support and implicit conversion semantics.

This structure preserves the principal semantic distinction made by BoostĖndian: **conversion functions and buffer types provide explicit conversion control, while arithmetic types provide convenience through implicit conversion and direct arithmetic operations**.

The implementation also demonstrates a reduction in unsafe code compared with a direct low-level C++ translation. The Rust-native implementation relies primarily on safe standard-library operations such as `swap_bytes()`, `to_be_bytes()`, `from_be_bytes()`, `to_le_bytes()`, and `from_le_bytes()`. Consequently, byte-order manipulation does not require manual pointer arithmetic or memory reinterpretation.

The C++ interoperability layer is kept separate through `ffi.rs` and the `cxx` bridge. This provides a defined boundary between the Rust implementation and existing C++ applications while retaining the original library's three conceptual usage models.

Finally, the implementation demonstrates that Boost.Endian's functionality can be reproduced without reproducing its C++ template hierarchy literally. The resulting design is instead based on Rust traits, generic types, type aliases, and operator traits, providing equivalent functionality while following the abstractions of the target language.

This is consistent with the migration strategy established for the project: the goal is to preserve relevant observable behaviour and functionality rather than mechanically reproduce the internal structure of the original Boost implementation. The dissertation identifies this as an explicit migration objective alongside C++ interoperability, Rust memory and type safety, acceptable performance, and isolation of unsafe code.

Important Scope Note

There are several differences between **Boost.Endian 1.91.0** and the current project implementation that should be made explicit rather than implying complete API equivalence:

- Boost supports **8-, 16-, 24-, 32-, 40-, 48-, 56- and 64-bit** buffer and arithmetic representations. The Rust implementation provides the 24-bit conversion functions, but its generic `EndianBuf` and `EndianInt` types operate on standard Rust primitive types rather than arbitrary bit widths.
- Boost provides both **aligned and unaligned** buffer and arithmetic variants. The Rust implementation does not reproduce this distinction as a separate generic alignment parameter.
- The Rust `cxx` bridge currently exposes the **16-, 32- and 64-bit** buffer and load/store interfaces. The 24-bit functionality remains available in the Rust conversion layer rather than being exposed through the demonstrated C++ bridge.
- The C++ arithmetic interface is implemented as a **C++ wrapper over the Rust bridge**, because `cxx` does not directly expose the Rust operator-trait implementation as a C++ operator-overloaded type.
- The repository's `endian_comparison.md` records that several implementation variants were explored during development. Therefore, benchmark and implementation-status statements should refer specifically to the variant being evaluated rather than treating all variants as a single final implementation.

These limitations are not necessarily shortcomings of the migration. Rather, they demonstrate that the project implements a **selected, semantically relevant subset of Boost.Endian**, while adapting the interface to Rust and preserving C++ interoperability where required. The original Boost documentation itself describes the three approaches as alternatives with different engineering trade-offs rather than requiring every application to use every facility.

Chapter 7

Conclusions and Future Work

7.1 Conclusions

This dissertation investigated an incremental approach for migrating selected low-level Boost functionality from C++ to Rust while preserving interoperability with existing C++ software. The work treated migration as an architectural and semantic transformation rather than as a direct source translation. Two case studies, `Boost.Align` and `Boost.Endian`, were implemented using different interoperability strategies so that the methodology could be evaluated under different compatibility constraints.

The study demonstrates that selected functionality from both libraries can be expressed using Rust-native abstractions while retaining a usable C++ integration boundary. The implementations also show that the appropriate FFI mechanism depends on the compatibility requirements of the component: the Align implementation uses a small C-compatible interface because the target use case retains C++03 compatibility, whereas the Endian implementation uses the `cxx` bridge.

The evaluation provides quantitative evidence for the selected workloads. In the Boost.Align end-to-end benchmark, the Rust implementation measured 20.7014 ms per iteration compared with 21.6782 ms for the Boost implementation, corresponding to approximately 4.7% lower measured execution time. The same experiment reported a larger RSS increase during allocation for the Rust implementation, so the results do not support a general claim of reduced memory consumption. In the Boost.Endian benchmark, Rust through the `cxx` bridge measured lower execution time than the manual C++ byte-manipulation baseline at all four tested input sizes, with a reported average speedup of approximately 35%. Because that baseline is not the original Boost.Endian implementation, the result should not be interpreted as evidence that Rust is generally faster than Boost.Endian.

The central conclusion is therefore methodological: successful incremental migration depends on identifying observable semantics, selecting an appropriate Rust abstraction, defining an explicit interoperability contract, isolating unsafe operations, and validating each migrated component with evidence appropriate to the property being claimed. The two case

studies demonstrate these patterns without establishing that every Boost component, particularly highly template-dependent components, can be migrated using the same implementation choices.

7.2 Answers to the Research Questions

7.2.1 RQ1 – Architectural and interoperability patterns

The implementation identifies several reusable patterns: semantic rather than syntactic translation, explicit ownership and lifetime contracts, small FFI surfaces, isolation of unsafe operations, compatibility-driven FFI selection, and validation against observable behaviour. These patterns are implemented in different forms for Align and Endian rather than through a single universal binding mechanism.

7.2.2 RQ2 – Component-selection criteria

The methodology defines selection criteria covering automotive relevance, safety relevance, isolation capability, interoperability constraints, portability, performance relevance, implementation complexity, and the potential to expose reusable migration patterns. These criteria provide a traceable basis for selecting case studies rather than selecting components solely because they are easy to translate.

7.2.3 RQ3 – Rust reimplementation while preserving behaviour

The two case studies demonstrate that selected Boost functionality can be re-expressed using Rust-native abstractions while preserving the tested observable behaviour. The implementations do not reproduce the complete Boost APIs or their internal C++ structures. Consequently, the evidence supports semantic reproduction of the selected functionality rather than complete API equivalence.

7.2.4 RQ4 – FFI boundary design

The implementations show two compatible approaches. Boost.Align uses a small C-compatible boundary suitable for C++03 consumers, while Boost.Endian uses `cxx`. In both cases, interface design requires explicit reasoning about representation, ownership, lifetime, valid inputs, and error or panic behaviour. The appropriate mechanism is therefore determined by the compatibility requirements of the component rather than selected independently of them.

7.2.5 RQ5 – Observed performance characteristics

The evaluated workloads show competitive measured execution times for the Rust implementations. Boost.Align measured approximately 4.7% lower execution time in the selected end-to-end AVX workload, while Boost.Endian measured approximately 35% lower execution

time than the manual C++ baseline across the tested sizes. These are workload-specific measurements and do not establish universal performance differences. In particular, the Align implementations did not use identical implementation details in every aspect, and the Endian baseline was not the original Boost.Endian implementation.

7.2.6 RQ6 – Safety guarantees and FFI contracts

Within safe Rust, ownership, borrowing, bounds, and representation invariants can be expressed and checked by the language and type system. At the FFI boundary, these guarantees become dependent on interface contracts and foreign-code behaviour. The caller must respect requirements concerning pointer validity, buffer length, alignment, lifetime, aliasing, and data layout. The migration therefore reduces the manually managed safety surface inside the Rust component but does not remove the need for verification of the hybrid boundary.

7.3 Contributions

The dissertation contributes a structured component-selection framework, a migration methodology, two complementary low-level case studies, an analysis of generalizable versus library-specific migration decisions, and an experimental evaluation of correctness, performance, memory observations, and FFI overhead. It also makes explicit the distinction between properties enforced by Rust internally and properties that remain contracts across FFI.

7.4 Limitations

The work is limited to selected functionality from two Boost libraries and therefore does not establish complete migration of either library or the feasibility of migrating the entire Boost ecosystem. The experiments were conducted on a limited set of workloads and hardware configurations. The performance measurements characterize observed execution time and throughput but do not establish worst-case execution time, jitter, schedulability, or hard real-time guarantees. The memory measurements are process-level RSS observations rather than a complete allocator or memory-efficiency analysis.

The safety discussion likewise should not be interpreted as a certification argument. The dissertation identifies where Rust's static guarantees apply and where explicit FFI contracts remain necessary, but it does not provide a complete ISO 26262 safety case, formal proof, or certification assessment of the resulting software.

Finally, the Endian performance comparison uses a manual C++ byte-manipulation baseline rather than a direct Boost.Endian implementation. The Align benchmark also contains implementation differences that limit the interpretation of the measured performance difference. These limitations are considered explicitly when drawing conclusions from the results.

7.5 Future Work

The work presented in this dissertation establishes a migration methodology and two implemented case studies, but it also identifies several directions for further development. These directions are deliberately limited to gaps that affect the completeness, robustness, or external validation of the current results. They should be understood as extensions of the present work, rather than as requirements for the conclusions reached in this dissertation.

7.5.1 Implementation Completion

For `Boost.Align`, the most relevant next steps are to generalise the current aligned-buffer abstraction, reduce the amount of low-level pointer handling exposed by the implementation, and improve the separation between generic aligned storage and the SIMD-specific processing demonstrated in the case study. Additional supported data types and explicit handling of runtime SIMD capabilities could also make the implementation applicable to a wider range of workloads.

For `Boost.Endian`, the main implementation gap is API coverage. Future work should consolidate the experimental implementation variants into one canonical implementation and extend support for the non-standard integer widths and buffer representations provided by Boost.Endian. The C++ interface should also be expanded where required, with an explicit distinction between the Rust API and the subset exposed through FFI.

7.5.2 Stronger Validation

The current experiments provide quantitative evidence for the selected workloads, but broader validation would improve the external validity of the results. In particular, future experiments should:

- compare against the original Boost.Endian implementation in addition to the manual C++ baseline;
- use equivalent operations and compiler settings for the C++ and Rust Align implementations;
- repeat measurements across additional hardware and toolchain configurations;
- report statistical variation in addition to average execution time;
- separate the cost of FFI calls from the cost of the underlying operation by processing buffers in larger batches; and
- validate binary layout, alignment and endian representations systematically through automated tests.

Cross-architecture testing would be particularly relevant for Endian because the implementation must distinguish external byte representation from the native representation of the host.

Similarly, Align should be evaluated on architectures and SIMD configurations beyond the environment used for the current benchmark.

These experiments would strengthen the evidence for portability and performance characteristics without changing the scope of the present conclusions.

7.5.3 FFI and Integration Refinement

The evaluation identified several improvements to the interoperability interfaces. FFI functions should preferably report invalid input through explicit error representations rather than relying on assertions or panic behaviour. Ownership, lifetime, buffer length, alignment and representation requirements should remain documented as part of each interface contract.

For performance-sensitive applications, buffer-oriented FFI should also be preferred over repeated per-element calls where practical. A single call processing a complete buffer would make the interface overhead easier to measure and reduce the number of boundary crossings.

Finally, the Rust and C++ interfaces should be tested as a complete integration rather than only through isolated examples. This includes compiler compatibility, build integration, ABI assumptions and regression tests for the observable behaviour used by existing C++ consumers.

7.5.4 Automotive-Oriented Validation

A natural continuation is to evaluate the migrated components in representative automotive-oriented workloads, such as sensor processing, binary protocol parsing, serialization, communication buffers, or other low-level data-processing paths. Eclipse S-CORE provides a relevant architectural context for this type of investigation because it targets modern automotive software architectures in which incremental coexistence of C++ and Rust components can be relevant.

Such an evaluation should focus on integration characteristics rather than assuming that the results of the present microbenchmarks transfer directly to a complete automotive system. In particular, system-level experiments could examine build integration, resource usage, interface frequency and the interaction of the migrated component with its surrounding software.

7.5.5 Extension to Additional Boost Components

The most important longer-term research direction is to apply the migration criteria and patterns identified in this dissertation to additional Boost components, especially components with substantially greater template complexity. Such work would test whether the patterns identified here remain useful when the amount of generic C++ structure and dependency coupling increases.

The selected case studies should therefore be viewed as a foundation for further investigation rather than as evidence that the same implementation strategy applies unchanged to every Boost library. Future studies can use the selection criteria and the distinction between general migration patterns and library-specific decisions established in this dissertation to determine which additional components are suitable candidates.

7.5.6 Summary

In summary, future work is concentrated in four areas: completing and stabilising the selected implementations, strengthening experimental validation, refining the FFI and automotive integration boundaries, and applying the methodology to additional Boost components. These extensions would broaden the evidence obtained in this dissertation while preserving its central scope: investigating an incremental and evidence-based approach to migrating selected low-level Boost functionality from C++ to Rust.

Bibliography

70% of Security Bugs Are Memory Safety Issues (2020). Tech. rep. Microsoft Security Response Center.

Amazon Web Services (2026). *Firecracker: Secure and Fast MicroVMs for Serverless Computing*. Firecracker main page. URL: https://firecracker-microvm.github.io/.

AUTOSAR Partnership (2023). *AUTOSAR Adaptive Platform - Explanation of Adaptive Platform Design*. Tech. rep. R23-11. Describes the service-oriented C++ architecture used in modern SDVs. AUTOSAR GbR. URL: https://www.autosar.org/standards/adaptive-platform.

Blandy, Jim, Jason Orendorff, and Leonora F.S. Tindall (2021). *Programming Rust: Fast, Safe Systems Development*. Sebastopol, CA: O'Reilly Media. ISBN: 978-1-492-05259-3. URL: https://oreilly.com.

Boost C++ Libraries Documentation (2025). URL: https://www.boost.org/docs/.

Bristol, Andrea (2025). *Ada and Rust are highlighted by the NSA and CISA in Memory Safe Language Information Sheet*. URL: https://www.adacore.com/blog/ada-and-rust-are-highlighted-by-the-nsa-and-cisa-in-memory-safe-language-information-sheet.

C++ ABI Compatibility (2025). Accessed: 2025-01-18. URL: https://parallel-rust-cpp.github.io/cpp_abi.html.

CppCon (2020). *CppCon 2020 Presentation Materials*. Cpp Conference. URL: https://github.com/CppCon/CppCon2020.

crates.io: The Rust Community's Crate Registry (2025). URL: https://crates.io/crates?sort=recent-downloads.

Dashdamirli, N. (2025). *Analyzing the Performance and Practicality of C, Rust, Python, and Lua Programming Languages for Developing Microcontroller Applications*. Technical report.

Deloitte (2024). *Software-defined vehicles: Global manufacturer readiness study*. URL: https://www.deloitte.com/ua/en/Industries/automotive/analysis/software-defined-vehicles.html.

Foundation, Eclipse (2025). *Eclipse Safe Open Vehicle Core (S-Core)*. URL: https://projects.eclipse.org/projects/automotive.score.

Google (2021). *High-level design of C++/Rust interop*. States impossibility of using C++ templates via standard bindgen/C FFI. URL: https://crubit.rs/design/design.html.

Google Android Team (2024). *Rust / C++ Interop in Android Platform*. Case study of large-scale hybrid C++/Rust integration in a production OS. URL: https://source.android.com/docs/setup/build/rust/building-rust-modules/cpp-interop.

Hong, J. (2023). “Improving Automatic C-to-Rust Translation with Static Analysis”. In: *2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*.

Immunant, Inc. (2025). *c2rust: Migrate C code to Rust*. Version 0.21.0. URL: https://github.com/immunant/c2rust.

ISO (2018). *ISO 26262:2018 Road vehicles – Functional safety*. Part 6: Product development at the software level. Geneva, Switzerland: International Organization for Standardization.

ISO 26262-6:2018 Road vehicles — Functional safety — Part 6: Product development at the software level (2018). Defines requirements for software unit testing and freedom from interference. Geneva, Switzerland: International Organization for Standardization. URL: https://www.iso.org/standard/68388.html.

ISO/IEC 14882:2020 Programming languages — C++ (Dec. 2020). Defines the C++20 standard, including modules, coroutines, concepts, and ranges. Geneva, Switzerland: International Organization for Standardization. URL: https://www.iso.org/standard/79358.html.

Khan, Mustafif (2023). *Rust for C++ Programmers: Learn how to embed Rust in C/C++ with ease*. London, UK: BPB Online. ISBN: 978-93-5551-355-7. URL: https://www.bpbonline.com.

Klabnik, Steve, Carol Nichols, and The Rust Project Developers (2017). *The Rust Programming Language*. San Francisco, CA: No Starch Press. ISBN: 978-1593278281. URL: https://doc.rust-lang.org/book/.

Kuo, Akhil (2024). *Rust for Blockchain Application Development: Use the Power of Rust to Develop Scalable, Robust, and Secure Blockchain Applications*. Birmingham, UK: Packt Publishing. ISBN: 978-1-83763-464-4. URL: https://www.packtpub.com/en-us/product/rust-for-blockchain-application-development-9781837634644.

Ling, M. et al. (2022). “In Rust We Trust: A Transpiler from Unsafe C to Safer Rust”. In: *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. Pittsburgh, PA, USA, pp. 354–355.

Memory Safety in Android and Chrome (2022). Tech. rep. Google Project Zero.

Meneely, A. et al. (2025). “Just Use Rust: A Best-Case Historical Study of Open Source Vulnerabilities in C”. In: *2025 IEEE/ACM 3rd International Workshop on Software Vulnerability Management (SVM)*.

Mozilla (2026). *Including Rust Code in Firefox*. Firefox Source Docs documentation. Mozilla. URL: https://firefox-source-docs.mozilla.org/build/buildsystem/rust.html.

Okawa, S. and S. Yamaguchi (2024). “A Performance Study on Rust and C Programs”. In: *2024 Twelfth International Symposium on Computing and Networking (CANDAR)*.

Rooney, M. P. and S. J. Matthews (2023). “Evaluating FFT Performance of the C and Rust Languages on Raspberry Pi Platforms”. In: *2023 IEEE International Conference on Cluster Computing (CLUSTER)*.

Rust Documentation (2025). Accessed: 2025-01-18. URL: https://docs.rs/.

S&P Global Mobility (2025). *The software-defined vehicle market is taking off*. URL: https://www.spglobal.com/automotive-insights/en/blogs/2025/09/the-software-defined-market-is-taking-off.

Stroustrup, Bjarne (2013). *The C++ Programming Language*. 4th ed. Addison-Wesley.

– (2020). “Thriving in a Crowded and Changing World: C++ 2006–2020”. In: *Proceedings of the ACM on Programming Languages* 4.HOPL, 70:1–70:168. DOI: 10.1145/3386320. URL: https://www.stroustrup.com/hopl20main-p5-p-bfc9cd4--final.pdf.

Synopsys (2025). *What is ASIL (Automotive Safety Integrity Level)?* Accessed: 2026-01-31. URL: https://www.synopsys.com/glossary/what-is-asil.html.

The Linux Kernel Organization (2026). *Rust — The Linux Kernel documentation*. Documentation for experimental Rust support in the Linux kernel (v6.1+). URL: https://docs.kernel.org/rust/index.html.

The MISRA Consortium (Oct. 2023). *MISRA C++:2023 Guidelines for the use of C++17 in critical systems*. Industry standard attempting to mitigate C++ memory safety risks via subsetting. MISRA Ltd. Nuneaton, UK. ISBN: 978-1-906400-33-3. URL: https://misra.org.uk/.

The Rust Project Developers (2026a). *Embedded Devices - Rust Programming Language*. Rust for embedded devices. URL: https://www.rust-lang.org/what/embedded.

– (2026b). *The Rustonomicon: Foreign Function Interface*. Accessed: 2026-09-26. URL: https://doc.rust-lang.org/nomicon/ffi.html.

The rust-bindgen Project Developers (2026). *Crate bindgen: Generate Rust bindings for C and C++ libraries*. Rust documentation. URL: https://docs.rs/bindgen/latest/bindgen/.

Tolnay, David (2025). *cxx: Safe Interop between Rust and C++*. The industry-standard library for safe, static-analysis-driven FFI between Rust and C++. URL: https://github.com/dtolnay/cxx.

Xia, L., Y. Wu, and B. Hua (2023). “Rustcheck: Safety Enhancement of Unsafe Rust via Dynamic Program Analysis”. In: *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*.

Zakarum (2023). *boosters: Rust port of boost libraries for C++*. Version 0.1.0. Rust crate version 0.1.0. URL: https://crates.io/crates/boosters.